

## 目录

|                                       |    |
|---------------------------------------|----|
| 1.机器学习模型 .....                        | 4  |
| 1.1 有监督学习模型 .....                     | 4  |
| 1.2 无监督学习模型 .....                     | 4  |
| 1.3 概率模型 .....                        | 5  |
| 1.4 什么是监督学习？什么是非监督学习？ .....           | 5  |
| 回归，分类，聚类方法的区别和联系并举例，简要介绍算法思路。 .....   | 5  |
| 生成模式 vs 判别模式 .....                    | 5  |
| 2.线性模型 .....                          | 6  |
| 2.1 线性回归 .....                        | 6  |
| 2.2 LR .....                          | 10 |
| 2.3 Lasso .....                       | 13 |
| 2.4 Ridge .....                       | 14 |
| 2.5 Lasso vs Ridge .....              | 14 |
| 2.6 线性回归 vs LR .....                  | 14 |
| 3.验证方式 .....                          | 14 |
| 3.1 什么是过拟合？产生过拟合原因？ .....             | 14 |
| 3.2 如何避免过拟合问题？ .....                  | 15 |
| 3.3 什么是机器学习的欠拟合？ .....                | 15 |
| 3.4 如何避免欠拟合问题？ .....                  | 15 |
| 3.5 什么是交叉验证？交叉验证的作用是什么？ .....         | 15 |
| 3.6 交叉验证主要有哪几种方法？ .....               | 15 |
| 3.7 什么是 K 折交叉验证？ .....                | 15 |
| 3.8 如何在 K 折交叉验证中选择 K？ .....           | 16 |
| 3.9 网格搜索（GridSearchCV） .....          | 16 |
| 3.10 随机搜索（RandomizedSearchCV） .....   | 16 |
| 4.分类 .....                            | 18 |
| 4.1 什么是准确率,精准率,召回率和 F1 分数？混淆矩阵 .....  | 18 |
| 4.2 模型常用的评估指标有哪些？ .....               | 18 |
| 4.3 多标签分类怎么解决？ .....                  | 19 |
| 5.正则化 .....                           | 19 |
| 手推 L1,L2 .....                        | 19 |
| 5.1 什么是正则化？如何理解正则化？ .....             | 20 |
| 5.2 L0、L1、L2 正则化？ .....               | 20 |
| 5.3 L1 和 L2 正则化有什么区别？ .....           | 20 |
| 5.4 L1 在 0 处不可导是怎么处理的？ .....          | 20 |
| 5.5 L1 正则化产生稀疏性的原因?对稀疏矩阵的理解？ .....    | 21 |
| 5.6 为何要常对数据做归一化？ .....                | 21 |
| 5.7 归一化的种类 .....                      | 21 |
| 5.8 归一化和标准化的区别 .....                  | 21 |
| 5.9 需要归一化的算法有哪些？这些模型需要归一化的主要原因？ ..... | 21 |
| 5.10 树形结构的不需要归一化的原因？ .....            | 22 |
| 6.特征工程 .....                          | 22 |
| 6.1 特征选择 .....                        | 22 |

|                                                             |    |
|-------------------------------------------------------------|----|
| 6.2 特征提取 .....                                              | 23 |
| 6.3 特征选择 vs 特征提取 .....                                      | 23 |
| 6.4 为什么要处理类别特征？怎么处理？ .....                                  | 23 |
| 6.5 什么是组合特征？ .....                                          | 23 |
| 6.6 怎么有效地找到组合特征？ .....                                      | 23 |
| 6.7 如何处理高维组合特征？ .....                                       | 23 |
| 6.8 如何解决数据不平衡问题？ .....                                      | 24 |
| 6.9 数据中有噪声如何处理？ .....                                       | 24 |
| 6.10 FM .....                                               | 24 |
| 6.11 FFM .....                                              | 25 |
| 7.决策树 .....                                                 | 25 |
| 7.1 ID3 算法 .....                                            | 25 |
| 7.2 C4.5 算法 .....                                           | 25 |
| 7.3 CART 算法 .....                                           | 26 |
| 7.4 ID3 vs C4.5 vs CART .....                               | 26 |
| 7.5 决策树 .....                                               | 27 |
| 熵(entropy) .....                                            | 29 |
| 7.6 信息增益 .....                                              | 30 |
| 7.7 信息增益率 .....                                             | 31 |
| 7.8 Hard Voting vs Soft Voting .....                        | 31 |
| 8.KNN .....                                                 | 32 |
| 8.1 简述一下 KNN 算法的原理? .....                                   | 32 |
| 8.2 如何理解 kNN 中的 k 的取值？ .....                                | 32 |
| 8.3 在 kNN 的样本搜索中，如何进行高效的匹配查找？ .....                         | 32 |
| 8.4 KNN 算法有哪些优点和缺点？ .....                                   | 32 |
| 8.5 不平衡的样本可以给 KNN 的预测结果造成哪些问题，有没有什么好的解决方式？ .....            | 33 |
| 8.6 为了解决 KNN 算法计算量过大的问题，可以使用分组的方式进行计算，简述一下该方式的原理。 .....     | 33 |
| 8.7 如何优化 Kmeans? .....                                      | 33 |
| 8.8 在 k-means 或 kNN，我们是用欧氏距离来计算最近的邻居之间的距离。为什么不用曼哈顿距离？ ..... | 33 |
| 8.9 参数说明以及调参 .....                                          | 33 |
| 9. SVM .....                                                | 34 |
| SVM 的推导 .....                                               | 34 |
| 9.1 SVM 的原理是什么？ .....                                       | 36 |
| 9.2 SVM 为什么采用间隔最大化？ .....                                   | 36 |
| 9.3 为什么 SVM 要引入核函数？ .....                                   | 36 |
| 9.4 为什么 SVM 对缺失数据敏感？ .....                                  | 36 |
| 9.5 SVM 核函数之间的区别 .....                                      | 36 |
| 9.6 SVM 如何处理多分类问题？ .....                                    | 37 |
| 9.7 带核的 SVM 为什么能分类非线性问题？ .....                              | 37 |
| 9.8 RBF 核一定是线性可分的吗？ .....                                   | 37 |
| 9.9 常用核函数及核函数的条件？ .....                                     | 37 |
| 9.10 为什么要将求解 SVM 的原始问题转换为对偶问题？ .....                        | 37 |

|                                                                                              |    |
|----------------------------------------------------------------------------------------------|----|
| 9.11 SVM 怎么输出预测概率？ .....                                                                     | 38 |
| 9.12 如何处理数据偏斜？ .....                                                                         | 38 |
| 9.13 LR vs SVM .....                                                                         | 38 |
| 9.14 参数说明 .....                                                                              | 38 |
| 9.15 LinearSVC vs SVC .....                                                                  | 39 |
| 10. 集成学习 .....                                                                               | 39 |
| 10.1 Boosting(提升法) .....                                                                     | 40 |
| 10.2 Bagging(套袋法) .....                                                                      | 48 |
| 10.3 Bagging vs Boosting .....                                                               | 50 |
| 10.4 随机森林 vs GBDT .....                                                                      | 51 |
| 10.5 AdaBoost vs GBDT .....                                                                  | 51 |
| 10.6 XGBoost vs GBDT .....                                                                   | 51 |
| 10.7 XGBoost vs LightGBM .....                                                               | 52 |
| 11. 无监督学习 .....                                                                              | 52 |
| 11.1 聚类 .....                                                                                | 52 |
| 11.2 降维 .....                                                                                | 54 |
| 12. 概率模型 .....                                                                               | 57 |
| 12.1 朴素贝叶斯 .....                                                                             | 57 |
| 12.2 朴素贝叶斯 vs LR .....                                                                       | 58 |
| 13. 机器学习 .....                                                                               | 59 |
| 13.1 你是怎么理解偏差和方差的平衡的？ .....                                                                  | 59 |
| 13.2 给你一个有 1000 列和 1 百万行的训练数据集，这个数据集是基于分类问题的。经理要求你来降低该数据集的维度以减少模型计算时间，但你的机器内存有限。你会怎么做？ ..... | 59 |
| 13.3 给你一个数据集，这个数据集有缺失值，且这些缺失值分布在离中值有 1 个标准偏差的范围内。百分之多少的数据不会受到影响？为什么？ .....                   | 59 |
| 13.4 模型受到低偏差和高方差问题时，应该使用哪种算法来解决问题呢？ .....                                                    | 60 |
| 13.5 怎么理解偏差方差的平衡的？ .....                                                                     | 60 |
| 13.6 协方差和相关性有什么区别？ .....                                                                     | 60 |
| 13.7 把分类变量当成连续型变量会更得到一个更好的预测模型吗？ .....                                                       | 60 |
| 13.8 机器学习中分类器指的是什么？ .....                                                                    | 60 |
| 13.10 请简要说说一个完整机器学习项目的流程？ .....                                                              | 60 |

# 一、机器学习

## 1.机器学习模型

### 1.1 有监督学习模型



### 1.2 无监督学习模型



## 1.3 概率模型



## 1.4 什么是监督学习？什么是非监督学习？

所有的回归算法和分类算法都属于监督学习。并且明确的给给出初始值，在训练集中有特征和标签，并且通过训练获得一个模型，在面对只有特征而没有标签的数据时，能进行预测。

**监督学习：**通过已有的一部分输入数据与输出数据之间的对应关系，生成一个函数，将输入映射到合适的输出，例如 分类。

**非监督学习：**直接对输入数据集进行建模，例如**强化学习**、**K-means 聚类**、**自编码**、**受限波尔兹曼机**。

**半监督学习：**综合利用有类标的数据和没有类标的数据，来生成合适的分类函数。

目前最广泛被使用的分类器有人工神经网络、支持向量机、最近邻居法、高斯混合模型、朴素贝叶斯方法、决策树和径向基函数分类。

无监督学习里典型的例子就是聚类了。聚类的目的在于把相似的东西聚在一起，一个聚类算法通常只需要知道如何计算相似度就可以开始工作了。

回归，分类，聚类方法的区别和联系并举例，简要介绍算法思路。

|           | 定义                                   | 算法                                     | 案例     |
|-----------|--------------------------------------|----------------------------------------|--------|
| <b>分类</b> | 对 <b>离散</b> 随机变量建模预测的监督学习算法          | LR、SVM、KNN、决策树、随机森林、GBDT               | 垃圾邮件分类 |
| <b>回归</b> | 对 <b>连续</b> 随机变量建模预测的监督学习算法          | 非线性回归、SVR（支持向量回归-->可用线性或高斯核（RBF））、随机森林 | 房价预测   |
| <b>聚类</b> | 基于数据的内部规律，寻找其属于不同族群的 <b>无监督学习</b> 算法 | Kmeans、层次聚类、GMM（高斯混合模型）、谱聚类            |        |

生成模式 vs 判别模式

**生成**模型：

由数据学得**联合概率分布函数  $P(X,Y)$** ，求出条件概率分布  $P(Y|X)$  的预测模型。

朴素贝叶斯、隐马尔可夫模型、高斯混合模型、文档主题生成模型（LDA）、限制波尔兹曼机。

**判别式**模型：

由数据直接学习决策函数  $Y = f(X)$ ，或由条件分布概率  $P(Y|X)$  作为预测模型。  
K 近邻、SVM、决策树、感知机、线性判别分析 (LDA)、线性回归、传统的神经网络、逻辑斯蒂回归、boosting、条件随机场。

## 2. 线性模型

### 2.1 线性回归

**原理:** 用线性函数拟合数据，用 **MSE** 计算损失，然后用梯度下降法(GD)找到一组使 MSE 最小的权重。

#### 线性回归的推导

|                                                                                                                                                                                                                                                 |                                                                                      |                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$                                                                                                                                                                                        | 化简:                                                                                  | $h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$                                                 |
| 拟合值跟实际值之间肯定会产生一个误差: ( $\epsilon$ )                                                                                                                                                                                                              |                                                                                      | $y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)}$                                                            |
| 误差具有独立同分布的特点，并且符合正态分布，其均值为0，方差为 $\sigma$ 的平方                                                                                                                                                                                                    |                                                                                      |                                                                                                          |
| 误差函数与正态分布的公式组合:                                                                                                                                                                                                                                 |                                                                                      | $p(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\epsilon^{(i)})^2}{2\sigma^2}\right)$ |
| $p(y^{(i)}   x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$                                                                                                                 |                                                                                      |                                                                                                          |
| 引入似然函数 $L(\theta) = \prod_{i=1}^m p(y^{(i)}   x^{(i)}; \theta) = \prod_{i=1}^m \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$                                                                  |                                                                                      |                                                                                                          |
| 函数是一个累乘的式子，将其取对数，将乘法变成加法，即为对数似然:                                                                                                                                                                                                                |                                                                                      |                                                                                                          |
| $\log L(\theta) = \log \prod_{i=1}^m \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$                       |                                                                                      |                                                                                                          |
| $\sum_{i=1}^m \log \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$<br>$= m \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{\sigma^2} \cdot \frac{1}{2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2$ |  | <b>最小二乘法</b><br>$J(\theta) = \frac{1}{2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2$                    |

|                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 接下来求 $J(\theta)$ 的 <b>极小值</b>                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                 |
| $J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2} (X\theta - y)^T (X\theta - y)$ |                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 求偏导:                                                                                                                 | $\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \left( \frac{1}{2} (X\theta - y)^T (X\theta - y) \right) = \nabla_{\theta} \left( \frac{1}{2} (\theta^T X^T - y^T) (X\theta - y) \right) \\ &= \nabla_{\theta} \left( \frac{1}{2} (\theta^T X^T X\theta - \theta^T X^T y - y^T X\theta + y^T y) \right) \\ &= \frac{1}{2} (2X^T X\theta - X^T y - (y^T X)^T) = X^T X\theta - X^T y \end{aligned}$ |
| 取偏导为0:<br>解得最终的参数值:                                                                                                  | $\theta = (X^T X)^{-1} X^T y$                                                                                                                                                                                                                                                                                                                                                                                   |

### 2.1.1 什么是回归？哪些模型可用于解决回归问题？

指分析因变量和自变量之间关系。

线性回归: 对异常值非常敏感

多项式回归: 如果指数选择不当，容易过拟合。

岭回归

Lasso 回归

弹性网络回归

### 2.1.2 线性回归的损失函数为什么是均方差？

|                                   |                                                                                                                                                                  |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 假设 $\epsilon$ 满足正态分布，所以其概率密度函数如下: | $f(\epsilon_i; u, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp \left[ -\frac{(\epsilon_i - u)^2}{2\sigma^2} \right]$                                       |
| 根据极大似然估计得定义:                      | $L(u, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \cdot \exp \left( -\frac{(\epsilon_i - u)^2}{2\sigma^2} \right)$                                     |
| 取似然对数得:                           | $\log L(u, \sigma^2) = -\frac{n}{2} \log \sigma^2 - \frac{n}{2} \log 2\pi - \frac{\sum_{i=1}^n (\epsilon_i - u)^2}{2\sigma^2}$                                   |
| 对 $\sigma^2$ 求偏导:                 | $\sigma^2 = \frac{1}{n} \sum_{i=1}^n (\epsilon_i - u)^2$                                                                                                         |
| 理论上是要求 $u=0$ ， $\sigma^2$ 越小越好    | $\sigma^2 = \frac{1}{n} \sum_{i=1}^n (\epsilon_i - u)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i - u)^2 \approx \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$ |

### 2.1.3 什么是线性回归？什么时候使用它？

利用最小二乘函数对一个或多个自变量和因变量之间关系进行建模的一种回归分析。

- (1) 自变量与因变量呈直线关系;
- (2) 因变量符合正态分布;
- (3) 因变量数值之间独立;
- (4) 方差是否齐性。

### 2.1.4 什么是梯度下降？SGD 的推导？

**BGD**：遍历全部数据集计算一次 loss 函数，然后算函数对各个参数的梯度，更新梯度。

|          |                                                                                                                                                                                                                                                                                                                                     |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 预测函数：    | $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$                                                                                                                                                                                                                                                     |
| 损失函数：    | $J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x) - y)^2$                                                                                                                                                                                                                                                                        |
| 求偏导：     | $\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 = (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y)$ $= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (\sum_{i=0}^n \theta_i x_i - y) = (h_{\theta}(x) - y) x_j$ |
| 梯度下降的公式： | $\theta_j := \theta_j - \alpha (h_{\theta}(x) - y) x_j$                                                                                                                                                                                                                                                                             |

|      | 名称                                                            | 训练样本                                  | 优点                                  | 缺点                                                        |
|------|---------------------------------------------------------------|---------------------------------------|-------------------------------------|-----------------------------------------------------------|
| BGD  | 批量梯度下降                                                        | 更新每一参数都用所有样本更新，m=all，更新100次遍历多有数据100次 | 耗时间长，但是一定能得到 <b>最优解</b>             |                                                           |
| SGD  | 随机梯度下降                                                        | 更新每一参数都随机选择一个样本更新，m=1                 | 迭代速度快，得到 <b>局部最优解</b> （凸函数时得到全局最优解） | 1. 需要一些超参数，如： <b>正则化参数</b> 和 迭代次数。<br>2. 对 <b>特征缩放</b> 敏感 |
| MBGD | 小批量梯度下降                                                       | 更新每一参数都选m个样本平均梯度更新，1<m<all            |                                     |                                                           |
| 总结：  | SGD训练速度快， <b>大样本</b> 选择；BGD能得到全局最优解， <b>小样本</b> 选择；MBGD综合二者选择 |                                       |                                     |                                                           |

| <b>SGDRegressor 实现思路</b>                                                                               |
|--------------------------------------------------------------------------------------------------------|
| 1. 先用train_test_split将数据随机分类成 训练数据和 测试数据；                                                              |
| 2. 使用StandardScaler类，fit方法将数据标准化，获取平均值和方差，标准差；                                                         |
| 3. 用同一标准(x的缩放方式x_scaler和y的缩放方式y_scaler)对训练数据和测试数据标准化, 具体标准化函数是： $\hat{x} = \frac{x - \bar{x}}{\sigma}$ |

### 2.1.5 什么是最小二乘法（最小平方法）？

它通过最小化误差的平方和寻找数据的最佳函数匹配。

### 2.1.6 常见的损失函数有哪些？

1.0-1 损失

2.均方差损失(MSE)

3.平均绝对误差(MAE)

4.分位数损失(Quantile Loss)

分位数回归可以通过给定不同的分位点，拟合目标值的不同分位数；  
实现了分别用不同的系数控制高估和低估的损失，进而实现分位数回归。

5.交叉熵损失

6.合页损失

一种二分类损失函数，SVM 的损失函数本质：**Hinge Loss + L2 正则化**  
合页损失的公式如下：

$$J_{hinge} = \sum_{i=1}^N \max(0, 1 - \text{sgn}(y_i) \hat{y}_i)$$

## 2.1.7 有哪些评估回归模型的指标？

衡量线性回归法最好的指标：**R Squared**

|                     | 公式                                                                                                                                                                                                      | 意义                    |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <b>MSE</b> (均方误差)   | $\frac{1}{m} \sum_{i=1}^m (y_{test}^{(i)} - \hat{y}_{test}^{(i)})^2$                                                                                                                                    | 值越大，表明预测效果越差          |
| <b>RMSE</b> (均方根误差) | $\sqrt{\frac{1}{m} \sum_{i=1}^m (y_{test}^{(i)} - \hat{y}_{test}^{(i)})^2}$                                                                                                                             | 放大了较大误差之间的差距；值越小其意义越大 |
| <b>MAE</b> (平均绝对误差) | $\frac{1}{m} \sum_{i=1}^m  y_{test}^{(i)} - \hat{y}_{test}^{(i)} $                                                                                                                                      | 反应的就是真实误差             |
| <b>R Squared</b>    | $R^2 = 1 - \frac{\sum_i (\hat{y}^{(i)} - y^{(i)})^2}{\sum_i (\bar{y} - y^{(i)})^2}$ <p>使用我们的模型预测产生的错误</p> <p>使用 <math>y = \bar{y}</math> 预测产生的错误</p> $= 1 - \frac{MSE(\hat{y}, y)}{Var(y)}$             |                       |
|                     | 1. $R^2 \leq 1$<br>2. $R^2$ 越大越好，当自己的预测模型不犯任何错误时： $R^2 = 1$<br>3. 当我们的模型等于基准模型时： $R^2 = 0$<br>4. 如果 $R^2 < 0$ ，说明学习到的模型还不如基准模型，很可能数据不存在任何线性关系。解决：用线性回归之前可以先对数据进行 <b>相关性检验</b> ，或者先对数据的 <b>残差分布</b> 进行判定 |                       |
|                     | $1 - \text{ourModelError} / \text{baselineModelError} = \text{我们模型拟合住的部分}$                                                                                                                              |                       |

## 2.1.8 什么是正规方程？

正规方程组是根据**最小二乘法**原理得到的关于参数估计值的线性方程组。

正规方程是通过求解  $\frac{\partial}{\partial \theta_j} J(\theta_j) = 0$  来找出使得代价函数最小的参数

解出： $\theta = (X^T X)^{-1} X^T y$

### 2.1.9 梯度下降法找到的一定是下降最快的方向吗？

不一定，它只是目标函数在当前的点的切平面上下降最快的方向。

在实际执行期中，牛顿方向（考虑海森矩阵）才一般被认为是下降最快的方向，可以达到超线性的收敛速度。梯度下降类的算法的收敛速度一般是线性甚至次线性的（在某些带复杂约束的问题）。

### 2.1.10 MBGD 需要注意什么？

如何选择  $m$ ？一般  $m$  取 2 的幂次方能充分利用矩阵运算操作。

一般会在每次遍历训练数据之前，先对所有的数据进行随机排序，然后在每次迭代时按照顺序挑选  $m$  个训练集数据直至遍历完所有的数据。

什么是正态分布？为什么要重视它？

如何检查变量是否遵循正态分布？

如何建立价格预测模型？价格是否正态分布？需要对价格进行预处理吗？

## 2.2 LR

也称为“对数几率回归”。

**知识点提炼**

1.分类，经典的二分类算法！

2.LR 的过程：面对一个回归或者分类问题，建立代价函数，然后通过优化方法迭代求解出最优的模型参数，然后测试验证这个求解的模型的好坏。

3.Logistic 回归虽然名字里带“回归”，但是它实际上是一种分类方法，主要用于两分类问题（即输出只有两种，分别代表两个类别）

4.回归模型中， $y$  是一个定性变量，比如  $y = 0$  或  $1$ ，logistic 方法主要应用于研究某些事件发生的概率。

5.LR 的本质：极大似然估计

6.LR 的激活函数：**Sigmoid**

7.LR 的代价函数：交叉熵

**优点：**

1.速度快，适合二分类问题

2.简单易于理解，直接看到各个特征的权重

3.能容易地更新模型吸收新的数据

**缺点：**

对数据和场景的适应能力有局限性，不如决策树算法适应性那么强。

LR 中最核心的概念是 Sigmoid 函数，Sigmoid 函数可以看成 LR 的激活函数。

Regression 常规步骤：

寻找  $h$  函数（即预测函数）

构造  $J$  函数（损失函数）

想办法（迭代）使得  $J$  函数最小并求得回归参数（ $\theta$ ）

LR **伪代码**：

初始化线性函数参数为 1

构造 sigmoid 函数

重复循环  $I$  次

    计算数据集梯度

更新线性函数参数  
 确定最终的 sigmoid 函数  
 输入训练（测试）数据集  
 运用最终 sigmoid 函数求解分类

## LR 的推导

|                                                                                                                                                                                                                                                                                                                                                                                                   |                                                 |                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|---------------------------------|
| LR模型的表达式:                                                                                                                                                                                                                                                                                                                                                                                         | $h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$ | $g'(x) = g(x) \cdot (1 - g(x))$ |
| 似然性的函数表达式: $L(\theta) = P(\vec{Y}   X; \theta) = \prod_{i=1}^N (h_{\theta}(x^{(i)})^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}})$                                                                                                                                                                                                                                                             |                                                 |                                 |
| $= \prod_{i=1}^N P(y^{(i)}   x^{(i)}; \theta)$                                                                                                                                                                                                                                                                                                                                                    |                                                 |                                 |
| 得对数似然函数: $l(\theta) = \sum_{i=1}^N \log l(\theta) = \sum_{i=1}^N y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))$                                                                                                                                                                                                                                                 |                                                 |                                 |
| 使用随机梯度下降的方法，对参数进行更新:                                                                                                                                                                                                                                                                                                                                                                              |                                                 |                                 |
| $\frac{\partial}{\partial \theta_j} l(\theta) = \left( y \frac{1}{h_{\theta}(x)} - (1 - y) \frac{1}{1 - h_{\theta}(x)} \right) \frac{\partial}{\partial \theta_j} h_{\theta}(x) = (y - h_{\theta}(x)) x_j$ $= \left( \frac{y(1 - h_{\theta}(x)) - (1 - y)h_{\theta}(x)}{h_{\theta}(x)(1 - h_{\theta}(x))} \right) h_{\theta}(x)(1 - h_{\theta}(x)) \frac{\partial}{\partial \theta_j} \theta^T x$ |                                                 |                                 |
| 通过扫描样本，迭代下述公式可解的参数: $\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$                                                                                                                                                                                                                                                                                                     |                                                 |                                 |

### 2.2.1 为什么 LR 要使用 sigmoid 函数？

1. 广义模型推导所得 2. 满足统计的最大熵模型 3. 性质优秀，方便使用  
 （Sigmoid 函数是平滑的，而且任意阶可导，一阶二阶导数可以直接由函数值得到不用进行求导，这在实现中很实用）

### 2.2.2 为什么常常要做特征组合（特征交叉）？

LR 模型属于线性模型，线性模型不能很好处理非线性特征，特征组合可以引入非线性特征，提升模型的表达能力。  
 另外，基本特征可以认为是全局建模，组合特征更加精细，是个性化建模，但对全局建模会对部分样本有偏，  
 对每一个样本建模又会导致数据爆炸，过拟合，所以基本特征+特征组合兼顾了全局和个性化。

### 2.2.3 为什么 LR 比线性回归要好？

LR 和线性回归首先都是广义的线性回归，  
 其次经典线性模型的优化目标函数是最小二乘，而 LR 则是似然函数，  
 另外线性回归在整个实数域范围内进行预测，敏感度一致，而分类范围，需要在 [0,1]。LR 就是一种减小预测范围，将预测值限定为 [0,1] 间的一种回归模型，因而对于这类问题来说，LR 的鲁棒性比线性回归的要好

## 2.2.4 LR 参数求解的优化方法？(机器学习中常用的最优化方法)

梯度下降法，随机梯度下降法，牛顿法，拟牛顿法（LBFGS，BFGS, OWLQN）  
目的都是求解某个函数的极小值。

## 2.2.5 工程上，怎么实现 LR 的并行化？有哪些并行化的工具？

LR 的并行化最主要的就是对目标函数梯度计算的并行化。

**无损的并行化：**算法天然可以并行，并行只是提高了计算的速度和解决问题的规模，但和正常执行的结果是一样的。

**有损的并行化：**算法本身不是天然并行的，需要对算法做一些近似来实现并行化，这样并行化之后的双方和正常执行的结果并不一致，但是相似的。

基于 Batch 的算法都是可以进行无损的并行化的。而基于 SGD 的算法都只能进行有损的并行化。

## 2.2.6 LR 如何解决低维不可分问题？

通过特征变换的方式把低维空间转换到高维空间，而在低维空间不可分的数据，到高维空间中线性可分的几率会高一些。

具体方法：**核函数**，如：高斯核，多项式核等等

## 2.2.7 LR 与最大熵模型 MaxEnt 的关系？

没有本质区别。LR 是最大熵对应类别为二类时的特殊情况，也就是当 LR 类别扩展到多类别时，就是最大熵模型。

## 2.2.8 为什么 LR 用交叉熵损失而不是平方损失（MSE）？

$$\hat{y} = \sigma(z), z = w \cdot x + b$$

如果使用**均方差**作为损失函数，求得的梯度受到 sigmoid 函数导数的影响：

$$C = \frac{(y - \hat{y})^2}{2} \quad \text{求导:} \quad \frac{\partial C}{\partial w} = (\hat{y} - y) \sigma'(z) x = (\hat{y} - y) \sigma'(z) x$$

如果使用**交叉熵**作为损失函数，没有受到 sigmoid 函数导数的影响，且真实值与预测值差别越大，梯度越大，更新的速度也就越快。

$$C = -\frac{1}{n} \sum [y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})] \quad \text{求导:} \quad \frac{\partial C}{\partial w} = \frac{1}{n} \sum x(\sigma(z) - y)$$

记忆：mse 的导数里面有 sigmoid 函数的导数，而**交叉熵导数里面没有 sigmoid 函数的导数**，sigmoid 的导数的最大值为 0.25，更新数据时太慢了。

## 2.2.9 LR 能否解决非线性分类问题？

可以，只要使用 **kernel trick**（核技巧）。

不过，通常使用的 kernel 都是隐式的，也就是找不到显式地把数据从低维映射到

高维的函数，而只能计算高维空间中数据点的内积。 $\sum_i a_i \langle x_i, x \rangle + b$

## 2.2.10 用什么来评估 LR 模型？

1. 由于 LR 是用来预测概率的，可以用 **AUC-ROC** 曲线以及混淆矩阵来确定其性能。
2. LR 中类似于校正 R2 的指标是 **AIC**。AIC 是对模型系数数量惩罚模型的拟合度量。因此，更偏爱有最小的 AIC 的模型。

### 2.2.11 LR 如何解决多分类问题？（OvR vs OvO）

|      | OvR                                                                                                                                    | OvO                                                                                                        |
|------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| 字面含义 | One vs Rest（一对剩余）                                                                                                                      | One vs One（一对一）                                                                                            |
| 算法思想 | n 种类型的样本进行分类时，分别取一种样本作为一类，将剩余的所有类型的样本看做另一类，这样就形成了 n 个二分类问题，使用逻辑回归算法对 n 个数据集训练出 n 个模型，将待预测的样本传入这 n 个模型中，所得概率最高的那个模型对应的样本类型即认为是该预测样本的类型； | n 类样本中，每次挑出 2 种类型，两两结合，一共有 $C_n^2$ 种二分类情况，使用 $C_n^2$ 种模型预测样本类型，有 $C_n^2$ 个预测结果，种类最多的那种样本类型，就认为是该样本最终的预测类型； |
| 区别   | OvO 用时较多，但其分类结果更准确                                                                                                                     |                                                                                                            |
| 使用   | <code>log_reg_ovr = LogisticRegression()</code><br>#默认支持多分类问题，分类方式为 'OvR'；                                                             | <code>log_reg_ovo = LogisticRegression(multi_class='multinomial', solver='newton-cg')</code>               |
| 图例解释 |                                                                                                                                        |                                                                                                            |

### 2.2.12 在训练的过程当中，如果有很多的特征高度相关或者说有一个特征重复了 100 遍，会造成怎样的影响？

如果在损失函数最终收敛的情况下，其实就算有很多特征高度相关也不会影响分类器的效果。但是对特征本身来说的话，假设只有一个特征，在不考虑采样的情况下，你现在将它重复 100 遍。训练以后完以后，数据还是这么多，但是这个特征本身重复了 100 遍，实质上将原来的特征分成了 100 份，每一个特征都是原来特征权重值的百分之一。如果在随机采样的情况下，其实训练收敛完以后，还是可以认为这 100 个特征和原来那一个特征扮演的效果一样，只是可能中间很多特征的值正负相消了。

### 2.2.13 为什么在训练的过程当中将高度相关的特征去掉？

去掉高度相关的特征会让模型的可解释性更好。

可以大大提高训练的速度。如果模型当中有很多特征高度相关的话，就算损失函数本身收敛了，但实际上参数是没有收敛的，这样会拉低训练的速度。其次是特征多了，本身就会增大训练的时间。

## 2.3 Lasso

定义：所有参数绝对值之和，即 L1 范数，对应的回归方法。

**Lasso(alpha=0.1)** # 设置学习率

$$L(x, y) \equiv \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1}^n |\theta_i|$$

## 2.4 Ridge

定义：所有参数平方和，即 **L2** 范数，对应的回归方法。

通过对系数的大小施加惩罚来解决 普通最小二乘法 的一些问题。

**Ridge(alpha=0.1)** # 设置惩罚项系数

$$L(x, y) \equiv \sum_{i=1}^n (y_i - h_{\theta}(x_i))^2 + \lambda \sum_{i=1}^n \theta_i^2$$

## 2.5 Lasso vs Ridge

|    |       | Lasso               | Ridge  |
|----|-------|---------------------|--------|
| 区别 | 正则化   | L1                  | L2     |
|    | 特征选择  | 可以                  | 不可以    |
|    | 求解方法  | 多种                  | 一种     |
|    | 鲁棒性   | 较好                  | 较差     |
|    | 贝叶斯角度 | 满足拉普拉斯分布            | 满足高斯分布 |
| 联系 |       | 都可以用来解决标准线性回归的过拟合问题 |        |
|    |       | 评价指标都是：R Squared    |        |

## 2.6 线性回归 vs LR

|    |      | 线性回归                                   | 逻辑回归                             |
|----|------|----------------------------------------|----------------------------------|
| 区别 | 构建方法 | 最小二乘法                                  | 似然函数                             |
|    | 解决问题 | 主要解决回归问题，也可以用来分类，但是鲁棒性差                | 解决分类问题                           |
|    | 输出   | 输出实数域上连续值                              | 输出值被S型函数映射到[0, 1], 通过设置阈值转换成分类类别 |
| 联系 |      | 都是广义上的线性回归，都是通过一系列输入特征拟合一条曲线来完成未知输入的预测 |                                  |

## 3.验证方式

### 3.1 什么是过拟合？产生过拟合原因？

指模型在训练集上的效果很好,在测试集上的预测效果很差.

1.数据有噪声

- 2.训练数据不足，有限的训练数据
- 3.训练模型过度导致模型非常复杂

### 3.2 如何避免过拟合问题？

|    |                |                       |
|----|----------------|-----------------------|
| 1  | early stopping | 在发生拟合之前提前结束训练。        |
| 2  | 数据集扩增          | 原有数据增加、原有数据加随机噪声、重采样。 |
| 3  | 正则化            | 引入范数概念，增强模型泛化能力。      |
| 4  | dropout        | 每次训练时丢弃一些节点，增强泛化能力    |
| 5  | 批量标准化          |                       |
| 6  | 减小模型复杂度        |                       |
| 7  | Bagging        |                       |
| 8  | 贝叶斯方法          |                       |
| 9  | 决策树剪枝          |                       |
| 10 | 集成方法           | 随机森林                  |

### 3.3 什么是机器学习的欠拟合？

模型复杂度低或者数据集太小,对模型数据的拟合程度不高,因此模型在训练集上的效果就不好.

### 3.4 如何避免欠拟合问题？

- 1.增加样本的数量
- 2.增加样本特征的个数
- 3.可以进行特征维度扩展
- 4.减少正则化参数
- 5.使用集成学习方法，如 Bagging

### 3.5 什么是交叉验证？交叉验证的作用是什么？

将原始 dataset 划分为两个部分.一部分为训练集用来训练模型,另外一部分作为测试集测试模型效果.

作用： 1)交叉验证是用来评估模型在新的数据集上的预测效果,也可以一定程度上减小模型的过拟合

2) 还可以从有限的数据中获取尽可能多的有效信息。

### 3.6 交叉验证主要有哪几种方法？

- ①留出法:简单地将原始数据集划分为训练集,验证集,测试集三个部分.
- ②k 折交叉验证:(一般取 5 折交叉验证或者 10 折交叉验证)
- ③LOO 留一法: (只留一个样本作为数据的测试集,其余作为训练集)---只适用于较少的数据集
- ④ Bootstrap 方法:(会引入样本偏差)

### 3.7 什么是 K 折交叉验证？

将原始数据集划分为  $k$  个子集，将其中一个子集作为验证集，其余  $k-1$  个子集作为训练集，如此训练和验证一轮称为一次交叉验证。

交叉验证重复  $k$  次，每个子集都做一次验证集，得到  $k$  个模型，加权平均  $k$  个模型的结果作为评估整体模型的依据。

### 3.8 如何在 $k$ 折交叉验证中选择 $k$ ?

$k$  越大，不一定效果越好，而且越大的  $k$  会加大训练时间；

在选择  $k$  时，需要考虑最小化数据集之间的方差，比如对于 2 分类任务，采用 2 折交叉验证，即将原始数据集对半分，若此时训练集中都是 A 类别，验证集中都是 B 类别，则交叉验证效果会非常差。

### 3.9 网格搜索 (GridSearchCV)

一种调优方法，在参数列表中进行穷举搜索，对每种情况进行训练，找到最优的参数。已 svm 调参为例：

```
from sklearn.model_selection import GridSearchCV

param_grid = {"gamma": [0.001, 0.01, 0.1, 1, 10, 100],
              "C": [0.001, 0.01, 0.1, 1, 10, 100]}
print("Parameters: {}".format(param_grid))

grid_search = GridSearchCV(SVC(), param_grid, cv=5)  # 5折交叉验证(分为5份)
x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, random_state=10)
grid_search.fit(x_train, y_train)

test_score = grid_search.score(x_test, y_test)
best_score = grid_search.best_score_
best_params = grid_search.best_params_

print("Test set score: {:.2f}".format(test_score))
print("Best parameters: {}".format(best_params))
print("Best score on train set: {:.2f}".format(best_score))

Parameters: {'gamma': [0.001, 0.01, 0.1, 1, 10, 100], 'C': [0.001, 0.01, 0.1, 1, 10, 100]}
Test set score: 0.97
Best parameters: {'C': 10, 'gamma': 0.1}
Best score on train set: 0.98
```

### 3.10 随机搜索 (RandomizedSearchCV)

```
boston_housing = boston_housing.load_data()
(x_train, y_train), (x_test, y_test) = boston_housing
x_train, x_valid, y_train, y_valid = train_test_split(
    x_train, y_train, random_state=6)

# 归一化
scaler = StandardScaler()
# 先拟合数据，再标准化
x_train_scaled = scaler.fit_transform(x_train)
x_valid_scaled = scaler.transform(x_valid)
x_test_scaled = scaler.transform(x_test)
```

```

# RandomizedSearchCV
# 1. 转化为 sklearn 的model
# 2. 定义参数集合
# 3. 开始搜索参数
def build_model(hidden_layers=1, layer_size=30,
                 learning_rates=3e-3):
    model = Sequential(Dense(layer_size, activation='relu',
                             input_shape=x_train[1:]))
    for _ in range(hidden_layers):
        model.add(Dense(layer_size, activation='relu'))
    model.add(Dense(1))
    optimizer = keras.optimizer.Adam(learning_rates)
    model.compile(loss='mse', optimizer=optimizer)
    return model

#把 tf model 转化为 sklearn model
kr_model = KerasRegressor(build_model)
#定义回调函数
log_dir = './search_housing_logs'
if not os.path.exists(log_dir):
    os.mkdir(log_dir)
save_model_dir = './search_housing_logs'
if not os.path.exists(save_model_dir):
    os.mkdir(save_model_dir)
save_model_file = os.path.join(save_model_dir, 'search_housing.h5')
#patience:在监测质量经过多少轮次没有进度时即停止
#min_delta:在被监测的数据中被认为是提升的最小变化
callbacks = [
    callbacks.TensorBoard(log_dir),
    callbacks.ModelCheckpoint(save_model_file, save_best_only=True),
    callbacks.EarlyStopping(patience=5, min_delta=1e-2)
]
param_random = {
    'hidden_layers': [1, 2, 3, 4, 5],
    'layer_size': np.arange(1, 100),
    'learning_rate': reciprocal(1e-4, 1e-2)
}
random_search_cv = RandomizedSearchCV(kr_model, param_random,
                                       n_iter=10, n_jobs=-1, cv=30)
random_search_cv.fit(x_train, y_train, epochs=20,
                    validation_data=(x_valid, y_valid),
                    callbacks=callbacks)

#查看做好的参数
print(random_search_cv.best_params_)
print(random_search_cv.best_score_)
print(random_search_cv.best_estimator_)
#获得最好的模型
model=random_search_cv.best_estimator_.model
model.evaluate(x_test, y_test)
# { 'hidden_layers' : 3, 'layer_size' : 32,
#   'learning_rate' : 0.0038406176766716676}

```

## 4.分类

### 4.1 什么是准确率,精准率,召回率和 F1 分数? 混淆矩阵

|      |   | 实际结果 |    |
|------|---|------|----|
|      |   | 1    | 0  |
| 预测结果 | 1 | TP   | FP |
|      | 0 | FN   | TN |

准确率 =  $(TP+TN)/\text{总样本数}$  = 预测正确的结果占总样本的百分比

### 4.2 模型常用的评估指标有哪些?

#### 4.2.1 Precision (查准率)

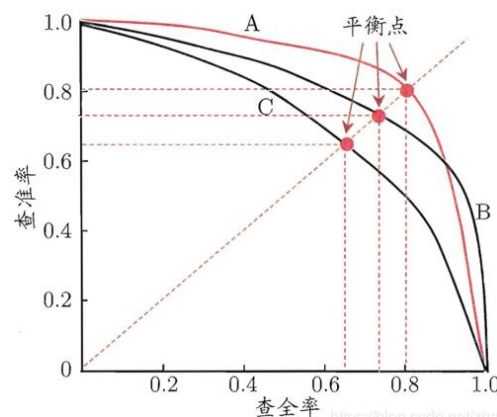
精准率(Precision) =  $TP/(TP+FP)$  = 所有被预测为正的样本中实际为正的样本的概率

#### 4.2.1 Recall (查全率)

召回率(Recall) =  $TP/(TP+FN)$  = 在实际为正的样本中被预测为正样本的概率

查准率和查全率是一对矛盾的度量, 一般而言, 查准率高时, 查全率往往偏低; 而查全率高时, 查准率往往偏低。

#### 4.2.3 P-R 曲线



横轴为召回率(查全率), 纵轴为精准率(查准率);

引入“平衡点”(BEP)来度量, 表示“查准率=查全率”时的取值, 值越大表明分类器性能越好。

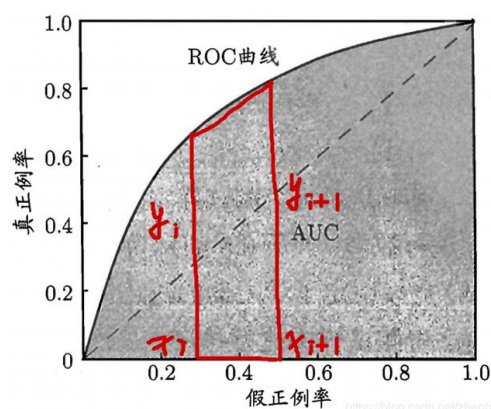
#### 4.2.4 F1-Score

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \quad \text{调和平均: } \frac{2}{F_1} = \frac{1}{P} + \frac{1}{R}$$

准确率和召回率的权衡: 只有在召回率 Recall 和精确率 Precision 都高的情况下, F1 score 才会很高, 比 BEP 更为常用。

### 4.2.5 ROC 和 AUC

#### 4.2.5.1 什么是 ROC 曲线? 如何判断 ROC 曲线的好坏?



ROC 曲线：横轴为 FPR，纵轴为 TPR

灵敏度  $TPR = TP/(TP+FN)$       特异度  $FPR = TN/(FP+TN)$

真正率 (**TPR**) = 灵敏度 =  $TP/(TP+FN) \iff$  真阳性率 = 召回 = TPR

假正率 (**FPR**) = 1 - 特异度 =  $FP/(FP+TN)$

**FPR** 的含义：所有确实为“假”的样本中，被误判真的样本。

TPR 越高，同时 FPR 越低（即 ROC 曲线越陡），那么模型的性能就越好。

#### 4.2.5.2 什么是 AUC?

$$AUC = \frac{1}{2} \sum_{i=1}^{m-1} (x_{i+1} - x_i) \cdot (y_i + y_{i+1})$$

AUC: ROC 曲线下的 面积，AUC 的取值范围在 0.5 和 1 之间。

衡量二分类模型优劣的一种评价指标，表示正例排在负例前面的概率。

#### 4.2.5.3 如何解释 AU ROC 分数?

表示预测准确性，AUC 值越高：预测准确率越高，反之越小 预测准确率越低。

AUC 如果小于 0.5，说明预测诊断比随机性猜测还差，实际情况中不应该出现这种情况，可能是设置的状态变量标准有误，需要查看设置。

### 4.3 多标签分类怎么解决?

问题转换

二元关联 (Binary Relevance)

分类器链 (Classifier Chains)

标签 Powerset (Label Powerset)

改编算法: kNN 的多标签版本是由 MLkNN 表示

集成方法: Scikit-Multilearn 库提供不同的组合分类功能

## 5.正则化

手推 L1,L2

|                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 模型权重 $w$ 可以看成是一个随机变量，符合某种分布，根据最大后验概率估计                                                                                                                              |
| $P(w X, y) = \frac{P(w, X, y)}{P(X, y)} = \frac{P(X, y w) P(w)}{P(X, y)} \propto P(y X, w) P(w)$                                                                    |
| 后面的 $P(w)$ 可以看成是一个先验和条件，取对数得：                                                                                                                                       |
| $\text{MAP} = \log P(y X, w) P(w) = \log P(y X, w) + \log P(w)$                                                                                                     |
| 对于后面的先验条件 $\log P(w)$ ，假设 $w$ 符合 <b>高斯分布</b> ，则：                                                                                                                    |
| $\log P(w) = \log \prod_j P(w_j) = \log \prod_j \left[ \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(w_j)^2}{2\sigma^2}} \right] = -\frac{1}{2\sigma^2} \sum_j w_j^2 + C'$ |
| 同理，假设 $w$ 符合 <b>拉普拉斯分布</b> ： $f(x) = \frac{1}{2\lambda} e^{-\frac{ x-\mu }{\lambda}}$ 均值： $\mu$ ，方差： $2\lambda^2$                                                   |
| $\log P(w) = \log \prod_j \frac{1}{\sqrt{2a}} e^{-\frac{ w_j }{a}} = -\frac{1}{a} \sum_j  w_j  + C'$                                                                |

### 5.1 什么是正则化？如何理解正则化？

定义：在损失函数后加上一个正则化项（**惩罚项**），其实就是常说的结构风险最小化策略，即**损失函数** 加上正则化。一般模型越复杂，**正则化值越大**。

正则化项是用来对模型中某些参数进行约束，正则化的一般形式：

$$\min \frac{1}{n} \sum L(y_i, f(x_i)) + \lambda J(f)$$

第一项是**损失函数**（经验风险），第二项是**正则化项**

公式可以看出，加上惩罚项后损失函数的值会增大，要想**损失函数最小**，**惩罚项**的值要**尽可能的小**，模型参数就要尽可能的小，这样就能减小模型参数，使得模型更加简单。

### 5.2 L0、L1、L2 正则化？

L0 范数：计算向量中非 0 元素的个数。

L0 范数和 L1 范数目的是使参数稀疏化。

L1 范数比 L0 范数容易优化求解。

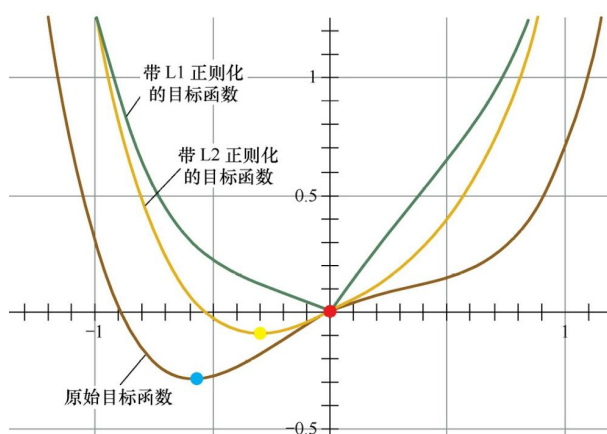
### 5.3 L1 和 L2 正则化有什么区别？

|      | L1正则                                         | L2正则         |
|------|----------------------------------------------|--------------|
| 定义   | 向量中各元素绝对值之和                                  | 向量中各元素平方和的开方 |
| 联系   | 降低损失函数                                       |              |
| 函数分布 | 拉普拉斯分布                                       | 高斯分布         |
| 作用   | 产生稀疏权值矩阵，即产生一个 <b>稀疏模型</b> ，可以用于 <b>特征选择</b> | 防止模型过拟合      |

### 5.4 L1 在 0 处不可导是怎么处理的？

- 1.坐标轴下降法是沿着坐标轴的方向  
Eg: **lasso** 回归的损失函数是不可导的
- 2.近端梯度下降(Proximal Algorithms)
- 3.交替方向乘子法(ADMM)

## 5.5 L1 正则化产生稀疏性的原因?对稀疏矩阵的理解?



L1 正则化会使得许多参数的**最优值**变成 0，这样模型就稀疏了。

**稀疏矩阵**指有很多元素为 0，少数参数为非零值。只有少部分特征对模型有贡献，大部分特征对模型没有贡献或者贡献很小，稀疏参数的引入，使得一些特征对应的参数是 0，所以就可以剔除可以将那些没有用的特征，从而实现特征选择，提高模型的泛化能力，降低过拟合的可能。

## 5.6 为何要常对数据做归一化?

- 1.归一化后加快的梯度下降对**最优解**的速度。
- 2.归一化有可能**提高精度**。

## 5.7 归一化的种类

| 类型     | 描述                                           | 公式                                           |
|--------|----------------------------------------------|----------------------------------------------|
| 线性归一化  | 利用max和min进行归一化，如果max和min不稳定，则常用经验值来替代max和min | $x' = \frac{x - \min(x)}{\max(x) - \min(x)}$ |
| 标准差归一化 | 利用所有样本的 <b>均值</b> 和 <b>方差</b> 将样本归一化为正态分布    | $x^* = \frac{x - \mu}{\sigma}$               |
| 非线性归一化 | 比如指数、对数、三角函数等                                |                                              |

## 5.8 归一化和标准化的区别

**标准化**是依照特征矩阵的列处理数据，其通过求 z-score 的方法，将样本的特征值转换到同一量纲下。归一化是依照特征矩阵的行处理数据，其目的在于样本向量在点乘运算或其他核函数计算相似性时，拥有统一的标准，也就是说都转化为“单位向量”。

归一化的目的是方便比较，可以加快网络的收敛速度；标准化是将数据利用 z-score（均值、方差）的方法转化为符合特定分布的数据，方便进行下一步处理，不为比较。

## 5.9 需要归一化的算法有哪些？这些模型需要归一化的主要原因？

线性回归,逻辑回归, KNN, SVM, 神经网络。

主要是因为特征值相差很大时,运用梯度下降,损失等高线是椭圆形,需要进行多次迭代才能达到最优点,如果进行归一化了,那么等高线就是圆形的,促使 SGD 往原点迭代,从而导致需要迭代次数较少。

### 5.10 树形结构的不需要归一化的原因?

因为它们不关心变量的值,而是关心变量分布和变量之间的条件概率,如决策树,随机森林;对于树形结构,树模型的构造是通过寻找最优分裂点构成的,样本点的数值缩放不影响分裂点的位置,对树模型的结构不造成影响,而且树模型不能进行梯度下降,因为树模型是阶跃的,阶跃是不可导的,并且求导没意义,也不需要归一化。

## 6.特征工程

特征工程分三步: ①数据预处理; ②特征选择; ③特征提取。

### 6.1 特征选择

特征选择在于选取对训练数据具有分类能力的特征,可以提高决策树学习的效率。通常特征选择的准则是信息增益或信息增益率。

特征选择的划分依据: 这一特征将训练数据集分割成子集,使得各个子集在当前条件下有最好的分类,那么就应该选择这个特征。

(将数据集划分为纯度更高,不确定性更小的子集的过程)

#### 6.1.1 什么是特征选择? 为什么需要它? 特征选择的目标?

指从已有的 M 个特征中选择 N 个特征使得系统的特定指标最优化,是从原始特征中选择出一些最有效特征以降低数据集维度的过程。

原因: ①减少特征数量、降维,使模型泛化能力更强,减少过拟合;

②增强对特征和特征值之间的理解。

且标: 选择离散程度高且与目标的相关性强的特征。

#### 6.1.2 有哪些特征选择技术?

##### ①过滤法

按照发散性或者相关性对各个特征进行评分,设定阈值或者待选择阈值的个数,从而选择特征;

##### ②包装法

根据目标函数(通常是预测效果评分),每次选择若干特征或者排除若干特征;常用方法主要是递归特征消除法。

##### ③嵌入法

先使用 ML 的算法和模型进行训练,得到各个特征的权重系数,根据系数从大到小选择特征;常用方法主要是基于惩罚项的特征选择法。



## 6.2 特征提取

常见的降维方法除了基于 **L1 惩罚项** 的模型以外，另外还有 **PCA** 和 **LDA**，**本质**是要将原始的样本映射到维度更低的样本空间中，但是它们的映射目标不一样：**PCA**是为了让映射后的样本具有最大的发散性；而 **LDA**是为了让映射后的样本有最好的分类性能。

## 6.3 特征选择 vs 特征提取

都是降维的方法。

特征选择：不改变变量的含义，仅仅只是做出筛选，留下对目标影响较大的变量；

特征提取：通过映射（变换）的方法，将高维的特征向量变换为低维特征向量。

## 6.4 为什么要处理类别特征？怎么处理？

除了决策树等少量模型能直接处理字符串形式的输入，对于 **LR, SVM** 等模型来说，类别特征必须经过处理转化成数值特征才能正常工作。方法主要有：

序号编码；独热编码；二进制编码

## 6.5 什么是组合特征？

为了提高复杂关系的拟合能力，在特征工程中经常会把一阶离散特征两两组合，构成高级特征。例如，特征 **a** 有 **m** 个取值，特征 **b** 有 **n** 个取值，将二者组合就有 **m\*n** 个组成情况。这时需要学习的参数个数就是 **m×n** 个。

## 6.6 怎么有效地找到组合特征？

可以用基于决策树的方法，首先根据样本的数据和特征构造出一颗决策树。然后从根节点到叶节点的每一条路径，都可以当作一种组合方式。

## 6.7 如何处理高维组合特征？

当每个特征都有千万级别，就无法学习  $m \times n$  规模的参数了。

解决：可以将每个特征分别用  $k$  维的低维向量表示，需要学习的参数变为  $m \times k + n \times k$  个，等价于矩阵分解

## 6.8 如何解决数据不平衡问题？

主要是由于数据分布不平衡造成的。

解决方法如下：

1. 采样，对小样本加噪声采样，对大样本进行下采样
2. 进行特殊的加权，如在 Adaboost 中或者 SVM 中
3. 采用对不平衡数据集不敏感的算法
4. 改变评价标准：用 AUC/ROC 来进行评价
5. 采用 Bagging/Boosting/ensemble 等方法
6. 考虑数据的先验分布

## 6.9 数据中有噪声如何处理？

噪声检查中比较常见的方法：

(1) 通过寻找数据集中与其他观测值及均值差距最大的点作为异常

(2) 聚类方法检测：将类似的取值组织成“群”或“簇”，落在“簇”集合之外的值被视为离群点。

在进行噪声检查后，通常采用分箱、聚类、回归、计算机检查和人工检查结合等方法“光滑”数据，去掉数据中的噪声。

采用分箱技术时，需要确定的两个主要问题就是：如何分箱以及如何对每个箱子中的数据进行平滑处理。

分箱的方法：有 4 种：等深分箱法、等宽分箱法、最小熵法和用户自定义区间法。

数据平滑方法

按平均值平滑：对同一箱值中的数据求平均值，用平均值替代该箱子中的所有数据。

按边界值平滑：用距离较小的边界值替代箱中每一数据。

按中值平滑：取箱子的中值，用来替代箱子中的所有数据。

## 6.10 FM

基于矩阵分解的机器学习算法,用于解决数据稀疏的业务场景（如推荐业务），特征怎样组合的问题。

如：一个广告分类的问题为例，根据用户与广告位的一些特征，来预测用户是否会点击广告。

| clicked | userId | userGender | itemTag |
|---------|--------|------------|---------|
| 1       | 1      | 男          | 篮球      |
| 0       | 1      | 男          | 化妆品     |
| 0       | 2      | 女          | 篮球      |
| 1       | 2      | 女          | 化妆品     |

对于 CTR 点击的分类预测中，有些特征是分类变量，一般进行 one-hot 编码。

One-hot 会带来数据的稀疏性，使得特征空间变大。

| clicked | userId | userGender_男 | userGender_女 | itemTag_篮球 | itemTag_化妆品 |
|---------|--------|--------------|--------------|------------|-------------|
| 1       | 1      | 1            | 0            | 1          | 0           |
| 0       | 1      | 1            | 0            | 0          | 1           |
| 0       | 2      | 0            | 1            | 1          | 0           |
| 1       | 2      | 0            | 1            | 0          | 1           |

### 6.10.1 SVM vs FM

- 1.SVM 的二元特征交叉参数是独立的，而 FM 的二元特征交叉参数是两个 k 维的向量  $v_i$ 、 $v_j$ ，交叉参数就不是独立的，而是相互影响的。
- 2.FM 可以在原始形式下进行优化学习，而基于 kernel 的非线性 SVM 通常需要在对偶形式下进行。
- 3.FM 的模型预测与训练样本独立，而 SVM 则与部分训练样本有关，即支持向量。

### 6.11 FFM

FM 在 FM 的基础上进一步改进，在模型中引入类别(field)的概念。将同一个 field 的特征单独进行 One-hot，因此在 FFM 中，每一维特征都会针对其他特征的每个 field，分别学习一个隐变量，该隐变量不仅与特征相关，也与 field 相关。FFM 中每一维特征都归属于一个特定和 field，field 和 feature 是一对多的关系：

| field   | user field(U) |              |              |                 |                 | item field(I) |             |
|---------|---------------|--------------|--------------|-----------------|-----------------|---------------|-------------|
| clicked | userId        | userGender_男 | userGender_女 | userAge_[20,30] | userAge_[30,40] | itemTag_篮球    | itemTag_化妆品 |
| 1       | 1             | 1            | 0            | 1               | 0               | 1             | 0           |
| 0       | 1             | 1            | 0            | 1               | 0               | 0             | 1           |
| 0       | 2             | 0            | 1            | 0               | 1               | 1             | 0           |
| 1       | 2             | 0            | 1            | 0               | 1               | 0             | 1           |

实现 FM & FFM 的最流行的 python 库有：LibFM、LibFFM、xlearn 和 tffm

名词解释： 点击率 CTR 转化率 CVR

## 7.决策树

### 7.1 ID3 算法

**核心**是在决策树各个节点上应用 **信息增益** 准则选择特征，递归的构建决策树。具体方法是：从根结点开始，对结点计算所有可能的特征的信息增益，选择**信息增益最大的特征**作为结点的特征，由该特征的不同取值建立子结点；再对子结点递归的调用以上方法，构建决策树；直到所有特征的信息增益均很小或没有特征可以选择为止。

ID3 相当于用**极大似然法**进行概率模型的选择。

使用 二元切分法 则易于对树构建过程中进行调整以处理连续型特征。

具体的处理方法是：如果特征值大于给定值就走左子树，否则走右子树。另外二元切分法也节省了树的构建时间。

### 7.2 C4.5 算法

算法用**信息增益率**选择特征，在树的构造过程中会进行剪枝操作优化，能够自动完成对连续属性的离散化处理；在选择分割属性时选择信息增益率最大的属性。

### 7.2.1 既然信息增益可以计算，为什么 C4.5 还使用信息增益比？

在使用信息增益的时候，如果某个特征有很多取值，使用这个取值多的特征会的大信息增益，这个问题是出现很多分支，将数据划分更细，模型复杂度高，出现过拟合的机率更大。使用**信息增益比**就是为了解决偏向于选择取值较多的特征的问题。使用信息增益比对取值多的特征加上的惩罚，对这个问题进行了校正。

## 7.3 CART 算法

分类与回归树 —— 使用**二元切分法**来处理连续型数值。

使用 Gini 作为分割属性选择的标准，择 **Gini 最大**的作为当前数据集的分割属性。

**Gini**: 表示在样本集合中一个随机选中的样本被分错的概率。

Gini 指数越小表示集合中被选中的样本被分错的概率越小，也就是说集合的纯度越高，反之，集合越不纯。

即 **基尼指数**（基尼不纯度）= 样本被选中的概率 \* 样本被分错的概率

$$\text{Gini}(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

**CART 算法**由以下两步组成：

**决策树生成**：基于训练数据集生成决策树，生成的决策树要尽量大；

**决策树剪枝**：用验证数据集对已生成的树进行剪枝并选择最优子树，这时用损失函数最小作为剪枝的标准。

### 7.3.1 基尼指数和信息熵都表示数据不确定性，为什么 CART 使用基尼指数？

信息熵 0, logK 都是值越大，数据的不确定性越大。信息熵需要计算对数，计算量大；信息熵是可以处理多个类别，基尼指数就是针对两个类计算的，由于 CART 树是一个二叉树，每次都是选择 yes or no 进行划分，从这个角度也是应该选择简单的基尼指数进行计算。

### 7.3.2 基尼系数(Gini)存在的问题？

基尼指数偏向于**多值**属性;当类数较大时，基尼指数求解比较困难;基尼指数倾向于支持在两个分区中生成大小相同的测试。

## 7.4 ID3 vs C4.5 vs CART

|            | ID3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | C4.5                                                  | CART                   |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|------------------------|
| 分叉情况       | 多叉树                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 多叉树                                                   | 二叉树                    |
| 应用类型       | 分类树                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 分类树                                                   | 分类和回归树                 |
| 生成过程特征是否复用 | 不复用                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 不复用(离散型特征)和复用(连续型特征)                                  | 复用                     |
| 特征选择方式     | 信息增益                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 信息增益比                                                 | 分类树: Gini<br>回归树: 平方误差 |
| 优点         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 产生的规则易于理解; 准确率较高; 实现简单;                               |                        |
| 缺点         | i1. 不能对连续数据进行处理, 只能通过连续数据离散化进行处理;<br>i2. 采用信息增益进行数据分裂容易偏向取值较多的特征, 准确性不如信息增益率;<br>i3. 缺失值不好处理;<br>i4. 没有采用剪枝, 决策树的结构可能过于复杂, 出现过拟合。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 1. 对数据进行多次顺序扫描和排序, 效率较低;<br>2. 只适合小规模数据集, 需要将数据放到内存中。 |                        |
| C4.5算法改进   | <p>i1: 将<b>连续的特征离散化</b>, 取相邻两样本值的平均值, 其中第<i>i</i>个划分点<math>T_i</math>表示: <math>T_i = \frac{a_i + a_{i+1}}{2}</math>, 分别计算以该点作为二元分类点的信息增益, 以信息增益最大的点作为该连续特征的二元离散分类点</p> <p>i2: 采用<b>信息增益率</b>的方法, 它是信息增益和特征熵的比值, 特征数越多的特征对应的特征熵越大, 它作为分母, 可以校正信息增益偏向取值较多的特征的问题</p> <p>i3: 主要需要解决的是两个问题,<br/>一是在样本某些特征缺失的情况下选择划分的属性:<br/>           对于第一个子问题, 对于某一个有缺失特征值的特征A, C4.5的思路是将数据分成两部分, 对每个样本设置一个权重(初始可以都为1), 然后划分数据, 一部分是有特征值A的数据D1, 另一部分是没有特征A的数据D2. 然后对于没有缺失特征A的数据集D1求和对应的A特征的所有特征值一起计算加权后的信息增益比, 最后乘上一个系数, 这个系数是无特征A缺失的样本加权后所占加权总样本的比例。</p> <p>二选定了划分属性, 对于在该属性上缺失特征的样本的处理:<br/>           对于第二个子问题, 可以将缺失特征的样本同时划分入所有的子节点, 不过将该样本的权重按各个子节点样本的数量比例来分配。比如缺失特征A的样本a之前权重为1, 特征A有3个特征值A1, A2, A3. 3个特征值对应的无缺失A特征的样本个数分别为2, 3, 4, 则a同时划分入A1, A2, A3. 对应权重调节为2/9, 3/9, 4/9。</p> <p>i4: 引入了正则化系数进行初步的<b>剪枝</b>, 剪枝有两种:<br/> <b>先剪枝</b>— 在构造过程中, 当某个节点满足剪枝条件, 则直接停止此分支的构造。<br/> <b>后剪枝</b>— 先构造完成完整的决策树, 再通过某些条件遍历树进行剪枝。</p> |                                                       |                        |

## 7.5 决策树

定义: 决策树就是一棵树, 其中跟节点和内部节点是输入特征的判定条件, 叶子结点就是最终结果。

其**损失函数**通常是 **正则化的极大似然函数**;

目标是 以损失函数为目标函数的最小化。

算法通常是一个 **递归**的选择最优特征, 并根据该特征对训练数据进行分割, 使得对各个子数据集有一个最好的分类过程。

决策树量化纯度:

判断数据集“纯”的指标有三个: **Gini 指数**、**熵**、**错误率**

### 7.5.1 决策树的数据 split 原理或者流程?

1. 将所有样本看做一个节点
  2. 根据纯度量化指标, 计算每一个特征的'纯度', 根据最不'纯'的特征进行数据划分
  3. 重复上述步骤, 知道每一个叶子节点都足够的'纯' 或者达到停止条件
- 背诵: 按照基尼指数、信息增益来选择特征, 保证划分后纯度尽可能高。

### 7.5.2 构造决策树的步骤?

1. 特征选择
2. 决策树的生成 (包含预剪枝) ---- 只考虑局部最优
3. 决策树的剪枝 (后剪枝) ---- 只考虑全局最优

### 7.5.3 决策树算法中如何避免过拟合和欠拟合?

**过拟合:** 选择能够反映业务逻辑的训练集去产生决策树;  
**剪枝操作** (前置剪枝和后置剪枝); **K 折交叉验证** (K-fold CV)  
**欠拟合:** 增加树的深度, RF

### 7.5.4 决策树怎么剪枝?

分为预剪枝和后剪枝, **预剪枝**是在决策树的构建过程中加入限制, 比如控制叶子节点最少的样本个数, 提前停止;

**后剪枝**是在决策树构建完成之后, 根据加上**正则项**的结构风险最小化自下向上进行的剪枝操作.

剪枝的**目的**就是**防止过拟合**, 是模型在测试数据上表现良好, 更加鲁棒.

### 7.5.5 决策树的优缺点?

决策树的**优点**:

1. 决策树模型可读性好, 具有描述性, 有助于人工分析;
2. 效率高, 决策树只需要一次性构建, 反复使用, 每一次预测的最大计算次数不超过决策树的深度。

决策树的**缺点**:

1. 即使做了预剪枝, 它也经常会过拟合, 泛化性能很差。
2. 对中间值的缺失敏感;
3. ID3 算法计算信息增益时结果偏向数值比较多的特征。

### 7.5.6 决策树和条件概率分布的关系?

决策树可以表示成给定条件下类的条件概率分布. 决策树中的每一条路径都对应是划分的一个条件概率分布. 每一个叶子节点都是通过多个条件之后的划分空间, 在叶子节点中计算每个类的条件概率, 必然会倾向于某一个类, 即这个类的概率最大.

### 7.5.7 为什么使用贪心和其发生搜索建立决策树, 为什么不直接使用暴力搜索建立最优的决策树?

决策树目的是构建一个与训练数据拟合很好, 并且复杂度小的决策树. 因为从所有可能的决策树中直接选择最优的决策树是 **NP 完全问题**, 在使用中一般使用启发式方法学习相对最优的决策树.

### 7.5.8 如果特征很多，决策树中最后没有用到的特征一定是无用吗？

不是无用的，从两个角度考虑，一是特征替代性，如果可以已经使用的特征 A 和特征 B 可以提点特征 C，特征 C 可能就没有被使用，但是如果把特征 C 单独拿出来进行训练，依然有效。其二，决策树的每一条路径就是计算条件概率的条件，前面的条件如果包含了后面的条件，只是这个条件在这棵树中是无用的，如果把条件拿出来也是可以帮助分析数据。

### 7.5.9 决策树怎么做回归？

给回归定义一个损失函数，比如 L2 损失，可以把分叉结果量化；最终的输出值，是分支下的样本均值。 [切分点选择：最小二乘法]；[输出值：单元内均值]。

### 7.5.10 决策树算法的停止条件？

#### 1.最小节点数

当节点的数据量小于一个指定的数量时，不继续分裂。两个原因：一是数据量较少时，再做分裂容易强化噪声数据的作用；二是降低树生长的复杂性。提前结束分裂一定程度上有利于降低过拟合的影响。

#### 2.熵或者基尼值小于阈值。

当熵或者基尼值过小时，表示数据的纯度比较大，如果熵或者基尼值小于一定程度数，节点停止分裂。

#### 3.决策树的深度达到指定的条件

决策树的深度是所有叶子节点的最大深度，当深度到达指定的上限大小时，停止分裂。

#### 4.所有特征已经使用完毕，不能继续进行分裂。

### 7.5.11 为什么决策树之前用 PCA 会好一点？

决策树的本质在于选取特征，然后分支。PCA 解除了特征之间的耦合性，并且按照贡献度给特征排了个序，这样更加方便决策树选取特征。

### 熵(entropy)

是表示随机变量不确定性的度量，是用来衡量一个随机变量出现的期望值。如果信息的不确定性越大，熵的值也就越大，出现的各种情况也就越多。

$$H(D) = - \sum_{k=1}^K \frac{|C_k|}{|D|} \log_2 \frac{|C_k|}{|D|}$$

条件熵( $H(Y | X)$ ): 表示在已知随机变量 X 的条件下随机变量 Y 的不确定性，其定义为 X 在给定条件下 Y 的条件概率分布的熵对 X 的数学期望:

$$entropy(D, A) = \sum_{i=1}^k \frac{D_{A_i}}{D} \log_2 D_{A_i}$$

### 二次代价函数

二次代价函数训练 NN，看到的实际效果是，如果误差越大，参数调整的幅度可能更小，训练更缓慢。

|              |                                                                                                                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 二次代价函数的公式如下： | $C = \frac{1}{2n} \sum_x \ y(x) - a^L(x)\ ^2$                                                                                                                                 |
| 以一个样本为例进行说明： | $C = \frac{(y - a)^2}{2} \rightarrow \begin{aligned} \frac{\partial C}{\partial w} &= (a - y)\sigma'(z)x \\ \frac{\partial C}{\partial b} &= (a - y)\sigma'(z) \end{aligned}$ |

## 交叉熵

用于度量两个概率分布间的差异性信息。语言模型的性能通常用交叉熵和复杂度来衡量。

交叉熵代价函数带来的训练效果往往比二次代价函数要好。

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 交叉熵代价函数     | $C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$                                                                                                                                                                                                                                                                                                                                                                                         |
| 求偏导         | $\begin{aligned} \frac{\partial C}{\partial w_j} &= -\frac{1}{n} \sum_x \left( \frac{y}{\sigma(z)} - \frac{(1 - y)}{1 - \sigma(z)} \right) \frac{\partial \sigma}{\partial w_j} \\ &= -\frac{1}{n} \sum_x \left( \frac{y}{\sigma(z)} - \frac{(1 - y)}{1 - \sigma(z)} \right) \sigma'(z) x_j \\ &= \frac{1}{n} \sum_x \frac{\sigma'(z) x_j}{\sigma(z)(1 - \sigma(z))} (\sigma(z) - y) \\ &= \frac{1}{n} \sum_x x_j (\sigma(z) - y) \end{aligned}$ |
| 同理可得，b的梯度为： | $\frac{\partial C}{\partial b} = \frac{1}{n} \sum_x (\sigma(z) - y)$                                                                                                                                                                                                                                                                                                                                                                             |

交叉熵代价函数是如何产生的？

|                                     |                                                                                                                                                                                                                                                                                                                                                                                |                                                                                 |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 以偏置b的梯度计算为例，推导出交叉熵代价函数：             | $\begin{aligned} \frac{\partial C}{\partial b} &= \frac{\partial C}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b} \\ &= \frac{\partial C}{\partial a} \cdot \sigma'(z) \cdot \frac{\partial (wx + b)}{\partial b} \\ &= \frac{\partial C}{\partial a} \cdot \sigma'(z) \\ &= \frac{\partial C}{\partial a} \cdot a(1 - a) \end{aligned}$ | <p>由二次代价函数推导出来的b的梯度公式为：</p> $\frac{\partial C}{\partial b} = (a - y)\sigma'(z)$ |
| 为了消掉该公式中的 $\sigma'(z)$ ，找到一个代价函数使得： | $\frac{\partial C}{\partial b} = (a - y) \quad \text{即：} \quad \frac{\partial C}{\partial a} \cdot a(1 - a) = (a - y)$                                                                                                                                                                                                                                                         |                                                                                 |
| 对两侧求积分，得：                           | $C = -[y \ln a + (1 - y) \ln(1 - a)] + \text{constant}$                                                                                                                                                                                                                                                                                                                        |                                                                                 |

## 7.6 信息增益

定义：特征 A 对训练数据集 D 的信息增益  $g(D, A)$ ，定义为集合 D 的经验熵  $H(D)$

与特征 A 给定条件下 D 的经验条件熵  $H(D|A)$  之差，即：

$$g(D,A)=H(D)-H(D|A)$$

信息增益：表示由于特征 A 使得对数据集 D 的分类的不确定性减少的程度。

信息增益 = entropy(前) - entropy(后)

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i) \quad Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j)$$

$$Gain(A) = Info(D) - Info_A(D)$$

特征选择方法是：对训练数据集 D，计算其每个特征的信息增益，并比较它们的大小，选择信息增益最大的特征。

### 7.6.1 为什么信息增益偏向取值较多的特征(缺点)?

当特征的取值较多时，根据此特征划分更容易得到纯度更高的子集，因此划分之后的熵更低，由于划分前的熵是一定的，因此信息增益更大，因此信息增益比较偏向取值较多的特征。

## 7.7 信息增益率

信息增益率 = 惩罚参数 \* 信息增益 （即信息增益和特征熵的比值）

$$g_R(D, A) = \frac{g(D, A)}{H_A(D)} \quad \text{其中:} \quad H_A(D) = - \sum_{i=1}^n \frac{|D_i|}{|D|} \log_2 \frac{|D_i|}{|D|}$$

信息增益比本质：是在信息增益的基础之上乘上一个惩罚参数。特征个数较多时，惩罚参数较小；特征个数较少时，惩罚参数较大。

惩罚参数：数据集 D 以特征 A 作为随机变量的熵的倒数，即：将特征 A 取值相同的样本划分到同一个子集中（数据集的熵是依据类别进行划分的）。

当特征取值较少时  $H_A(D)$  的值较小，因此其倒数较大，因而信息增益比较大。因而偏向取值较少的特征。

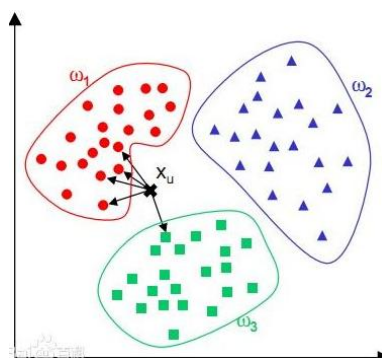
### 7.7.1 如何使用信息增益比?

在候选特征中找出信息增益高于平均水平的特征，然后在这些特征中再选择信息增益率最高的特征。

## 7.8 Hard Voting vs Soft Voting

|                       | Hard Voting                                                                                                                                                                                                                                                                                                                                                                                                                            | Soft Voting                                        |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| 思想                    | 根据少数服从多数来定最终结果；                                                                                                                                                                                                                                                                                                                                                                                                                        | 将所有模型预测样本为某一类别的概率的平均值作为标准，概率最高的对应的类型为最终的预测结果；      |
| 使用方式                  | voting = 'hard': 表示最终决策方式为 Hard Voting Classifier;                                                                                                                                                                                                                                                                                                                                                                                     | voting = 'soft': 表示最终决策方式为 Soft Voting Classifier; |
| scikit-learn 中使用集成分类器 | <pre> X, y = datasets.make_moons(n_samples=500, noise=0.3, random_state=42) X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)  voting_clf = VotingClassifier(estimators=[     ('log_clf', LogisticRegression()),     ('svc_clf', SVC(probability=False)),     ('dt_clf', DecisionTreeClassifier(random_state=666)) ], voting='hard') voting_clf.fit(X_train, y_train) voting_clf.score(X_test, y_test) </pre> |                                                    |
|                       | 使用 <u>Soft Voting</u> 时, SVC() 算法的参数: <code>probability=True</code>                                                                                                                                                                                                                                                                                                                                                                    |                                                    |

## 8.KNN



### 8.1 简述一下 KNN 算法的原理?

利用训练数据集对特征向量空间进行划分。KNN 算法的核心思想是在一个含未知样本的空间，可以根据样本最近的  $k$  个样本的数据类型来确定未知样本的数据类型。该算法涉及的 3 个主要因素是： $k$  值选择，距离度量，分类决策。

### 8.2 如何理解 kNN 中的 $k$ 的取值?

在应用中， $k$  值一般取比较小的值，并采用交叉验证法进行调优。

### 8.3 在 kNN 的样本搜索中，如何进行高效的匹配查找?

线性扫描(数据多时，效率低) 构建数据索引——Clipping 和 Overlapping 两种。前者划分的空间没有重叠，如  $k$ -d 树；后者划分的空间相互交叠，如 R 树。（对 R 树了解很少，可以之后再去了解）

### 8.4 KNN 算法有哪些优点和缺点?

|     | 优点                             | 缺点             |
|-----|--------------------------------|----------------|
| KNN | 既可以做分类也可以做回归                   | 计算量大           |
|     | 可以用于非线性分类/回归；训练时间复杂度为 $O(n)$ ； | 存在类别不平衡问题      |
|     | 准确率高，对数据没有假设，对离群点不敏感           | 需要大量的内存，空间复杂度高 |

### 8.5 不平衡的样本可以给 KNN 的预测结果造成哪些问题，有没有什么好的解决方式？

输入实例的  $k$  邻近点中，大数量类别的点会比较多，但其实可能都离实例较远，这样会影响最后的分类。

可以使用权值来改进，距实例较近的点赋予较高的权值，较远的赋予较低的权值。

### 8.6 为了解决 KNN 算法计算量过大的问题，可以使用分组的方式进行计算，简述一下该方式的原理。

先将样本按距离分解成组，获得质心，然后计算未知样本到各质心的距离，选出距离最近的一组或几组，再在这些组内引用 KNN。

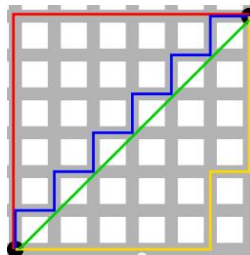
本质上就是事先对已知样本点进行剪辑，事先去除对分类作用不大的样本，该方法比较适用于样本容量比较大时的情况。

### 8.7 如何优化 Kmeans？

使用 Kd 树或者 Ball Tree：将所有的观测实例构建成一棵 kd 树，之前每个聚类中心都是需要和每个观测点做依次距离计算，现在这些聚类中心根据 kd 树只需要计算附近的一个局部区域即可。

### 8.8 在 k-means 或 kNN，我们是用欧氏距离来计算最近的邻居之间的距离。为什么不用曼哈顿距离？

曼哈顿距离只计算水平或垂直距离，有维度的限制。另一方面，欧氏距离可用于任何空间的距离计算问题。



绿色的线为欧式距离的丈量长度，红色的线即为曼哈顿距离长度，蓝色和黄色的线是这两点间曼哈顿距离的等价长度。

欧式距离：两点之间的最短距离；

曼哈顿距离：投影到坐标轴的长度之和；又称为出租车距离。

切比雪夫距离：各坐标数值差的最大值；

### 8.9 参数说明以及调参

**n\_neighbors**: 邻居节点数量

**weights**: 设为 distance（离一个簇中心越近的点，权重越高）；

**p=1** 为曼哈顿距离，**p=2** 为欧式距离。默认为 2

**leaf\_size**: 传递给 BallTree 或者 KDTree，表示构造树的大小，默认值是 30

**n\_jobs**: 并发执行的 job 数量，用于查找邻近的数据点。默认值 1，选取-1

占据 CPU 比重会减小，但运行速度也会变慢。

```
from sklearn import datasets
digits = datasets.load_digits()
x_train, x_test, y_train, y_test =
    train_test_split(digits.data, digits.target, random_state=10)
param_grid = [
    {
        'weights': ['distance'],
        'n_neighbors': [i for i in range(1,6)],
        'p': [i for i in range(1,6)]
    }
]
knn = KNeighborsClassifier()
grid_search = GridSearchCV(knn, param_grid, n_jobs=1)
grid_search.fit(x_train, y_train)
test_score = grid_search.score(x_test, y_test)
best_score = grid_search.best_score_
best_params = grid_search.best_params_
print("Test score: {:.2f}".format(test_score))
print("Best parameters: {}".format(best_params))
print("Best score : {:.2f}".format(best_score))

Test score:0.99
Best parameters: {'n_neighbors': 3, 'p': 2, 'weights': 'distance'}
Best score :0.98
```

## 9. SVM

SVM 又叫最大间隔分类器，最早用来解决二分类问题。

SVM 有三宝，间隔，对偶，核技巧

多项式核函数  $K(x, y) = (x \cdot y + c)^d$     线性核函数  $K(x, y) = x \cdot y$

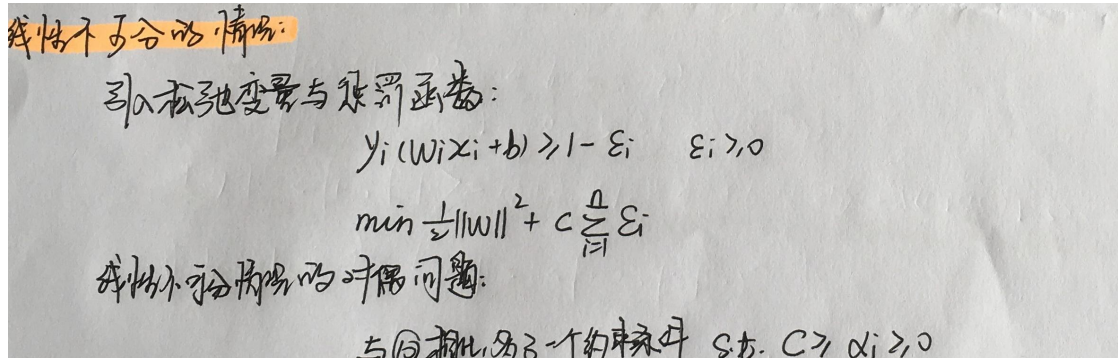
1. **SVC(kernel = 'poly')**：表示算法使用多项式核函数；
2. **SVC(kernel = 'rbf')**：表示算法使用高斯核函数；

SVM 算法的本质就是求解目标函数的最优化问题：

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \zeta_i$$

**SVM 的推导**





## 9.1 SVM 的原理是什么？

SVM 是一种二类分类模型。它的基本模型是在特征空间中寻找间隔最大化的分离超平面的线性分类器。（间隔最大是它有别于感知机）

（1）当训练样本线性可分时，通过硬间隔最大化，学习一个线性分类器，即线性可分支持向量机；

（2）当训练数据近似线性可分时，引入松弛变量，通过软间隔最大化，学习一个线性分类器，即线性支持向量机；

（3）当训练数据线性不可分时，通过使用核技巧及软间隔最大化，学习非线性支持向量机。

注：以上各 SVM 的数学推导应该熟悉：硬间隔最大化（几何间隔）---学习的对偶问题---软间隔最大化（引入松弛变量）--- 非线性支持向量机（核技巧）

## 9.2 SVM 为什么采用间隔最大化？

当训练数据线性可分时，存在无穷个分离超平面可以将两类数据正确分开。感知机利用误分类最小策略，求得分离超平面，不过此时的解有无穷多个。线性可分支持向量机利用间隔最大化求得最优分离超平面，这时，解是唯一的。另一方面，此时的分隔超平面所产生的分类结果是最鲁棒的，对未知实例的泛化能力最强。可以借此机会阐述一下几何间隔以及函数间隔的关系。

## 9.3 为什么 SVM 要引入核函数？

原始空间线性不可分，可以使用一个非线性映射将原始数据  $x$  变换到另一个高维特征空间，在这个空间中，样本变得线性可分。

解决方法：常用的一般是径向基 RBF 函数（线性核，高斯核，拉普拉斯核等）

## 9.4 为什么 SVM 对缺失数据敏感？

这里说的缺失数据是指缺失某些特征数据，向量数据不完整。SVM 没有处理缺失值的策略。而 SVM 希望样本在特征空间中线性可分，所以特征空间的好坏对 SVM 的性能很重要。缺失特征数据将影响训练结果的好坏。

## 9.5 SVM 核函数之间的区别

一般选择线性核和高斯核，也就是线性核与 RBF 核。

高斯核函数（RBF）：

$$K(x, y) = e^{-\gamma \|x - y\|^2}$$

本质：将每一个样本点映射到一个**无穷维**的特征空间。  
 高斯核**升维**的本质，使得线性不可分的数据**线性可分**。

| 模型     |             | 线性核         | RBF 核                                             |
|--------|-------------|-------------|---------------------------------------------------|
| 区别     |             | 用于线性可分      | 参数少，速度快                                           |
|        |             | 用于线性不可分     | 参数多，分类结果非常依赖于参数;很多人通过训练数据的交叉验证来寻找合适的参数，不过这个过程比较耗时 |
| 怎么选模型? | Feature 的数量 | 很大，跟样本数量差不多 | 选用 <b>线性核</b> 的 SVM                               |
|        |             | 比较小，样本数量一般  | 选用 <b>高斯核</b> 的 SVM                               |
|        |             | 比较小，而样本数量很多 | 需要手工添加一些feature变成第一种情况                            |

## 9.6 SVM 如何处理多分类问题？

一般有两种做法：一种是**直接法**，直接在目标函数上修改，将多个分类面的参数求解合并到一个最优化问题里面。看似简单但是计算量却非常的大。另外一种做法是**间接法**：对训练器进行组合。其中比较典型的有一对一，和一对多。一对多，就是对每个类都训练出一个分类器，由 **svm** 是二分类，所以将此而分类器的两类设定为目标类为一类，其余类为另外一类。这样针对  $k$  个类可以训练出  $k$  个分类器，当有一个新的样本来的时候，用这  $k$  个分类器来测试，那个分类器的概率高，那么这个样本就属于哪一类。这种方法效果不太好，**bias** 比较高。**svm** 一对一法（**one-vs-one**），针对任意两个类训练出一个分类器，如果有  $k$  类，一共训练出  $C(2,k)$  个分类器，这样当有一个新的样本要来的时候，用这  $C(2,k)$  个分类器来测试，每当被判定属于某一类的时候，该类就加一，最后票数最多的类别被认定为该样本的类。

## 9.7 带核的 SVM 为什么能分类非线性问题？

核函数的**本质**是两个函数的内积，而这个函数在 **SVM** 中可以表示成对于输入值的高维映射。注意核并不是直接对应映射，核只不过是一个内积

## 9.8 RBF 核一定是线性可分的吗？

不一定，RBF 核比较难调参而且容易出现**维度灾难**，要知道无穷维的概念是从泰勒展开得出的。

## 9.9 常用核函数及核函数的条件？

线性核：主要用于线性可分的情况

多项式核

RBF 核径向基，这类函数取值依赖于特定点间的距离，所以拉普拉斯核其实也是径向基核。

傅里叶核

样条核

Sigmoid 核函数

## 9.10 为什么要将求解 SVM 的原始问题转换为对偶问题？

1.对偶问题将原始问题中的约束转为了对偶问题中的等式约束；  
 2.方便核函数的引入；  
 3.改变了问题的复杂度。由求特征向量  $w$  转化为求比例系数  $a$ ，在原始问题下，求解的复杂度与样本的维度有关，即  $w$  的维度。在对偶问题下，只与样本数量有关。**对偶问题**是凸优化问题，可以进行较好的求解，SVM 中就是将原问题转换为对偶问题进行求解。

### 9.11 SVM 怎么输出预测概率？

把 SVM 的输出结果当作  $x$  经过一层 LR 模型得到概率，其中 lr 的  $w$  和  $b$  参数为使得总体交叉熵最小的值。

### 9.12 如何处理数据偏斜？

可以对数量多的类使得惩罚系数  $C$  越小表示越不重视，相反另数量少的类惩罚系数变大。

### 9.13 LR vs SVM

|    |        | 逻辑回归                             | SVM                  |
|----|--------|----------------------------------|----------------------|
| 区别 | 参数模型   | 是                                | 非                    |
|    | 数据集    | 较大                               | 较小                   |
|    | 损失函数   | 对数损失                             | 合页损失                 |
|    | 受类别平衡  | 影响                               | 不影响                  |
|    | 依赖数据分布 | 是                                | 否                    |
|    | 考虑损失   | 考虑了所有点的损失，但通过非线性操作大大减小离超平面较远点的权重 | 仅考虑支持向量的损失           |
|    |        |                                  | 依赖penalty系数，实验中需要做CV |
| 联系 |        | 1. 处理分类问题，且一般都用于处理线性二分类问题        |                      |
|    |        | 2. 增加不同的正则化项，如l1、l2              |                      |
|    |        | 3. 可以用来做非线性分类，需要加核函数             |                      |
|    |        | 4. 都是线性模型，都属于判别模型                |                      |

### 9.14 参数说明

```
SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,
    decision_function_shape='ovr', degree=3, gamma='auto', kernel='linear',
    max_iter=-1, probability=False, random_state=None, shrinking=True,
    tol=0.001, verbose=False)
```

- (1) `C`: 惩罚系数 $C$ , 用来平衡分类间隔和错分样本的, 默认1.0;  
 $C$ 越大, 容错空间越大;
  - (2) `kernel`: 参数选择有RBF, Linear, Poly, Sigmoid, 默认"RBF";
  - (3) `degree`: 多项式的最高次幂; 默认为3
  - (4) `gamma`: 核函数的系数('Poly', 'RBF' and 'Sigmoid'),  
默认:`gamma = 1/n_features`;
  - (5) `coef0`: 核函数中的独立项, 'RBF' and 'Poly'有效;
  - (6) `probability`: 可能性估计是否使用(true or false);
  - (7) `shrinking`: 是否进行启发式;
  - (8) `tol` (default =  $1e - 3$ ): svm结束标准的精度;
  - (9) `cache_size`: 制定训练所需要的内存(以MB为单位);
  - (10) `class_weight`: 每个类所占据的权重, 不同的类设置不同的惩罚参数 $C$ , 缺省的话自适应;
  - (11) `verbose`: 跟多线程有关;
  - (12) `max_iter`: 最大迭代次数, default = 1;
  - (13) `decision_function_shape`: 'ovo' 一对一, 'ovr' 多对多  
or None 无, default=None
- 提示: 7,8,9一般不考虑。

```
LinearSVC(C=1.0, class_weight=None, dual=True, fit_intercept=True,
          intercept_scaling=1, loss='squared_hinge', max_iter=1000,
          multi_class='ovr', penalty='l2', random_state=None, tol=0.0001,
          verbose=0)
```

## 9.15 LinearSVC vs SVC

|                 | LinearSVC               | SVC     |
|-----------------|-------------------------|---------|
| 多种惩罚参数和损失函数可供选择 | 有                       | 无       |
| 训练集实例数量大(大于1万)  | 可以很好地进行归一化              | 很难进行归一化 |
|                 | 既支持稠密输入矩阵也支持稀疏输入矩阵      |         |
| 联系              | 多分类问题采用 <b>OVR</b> 方法实现 |         |

## 线性回归 vs LR vs SVM:

线性回归做分类因为考虑了所有样本点到分类决策面的距离, 所以在两类数据分布不均匀的时候将导致误差非常大;

LR 和 SVM 克服了这个缺点, 其中 LR 将所有数据采用 `sigmoid` 函数进行了非线性映射, 使得远离分类决策面的数据作用减弱;

SVM 直接去掉了远离分类决策面的数据, 只考虑支持向量的影响。

## 10. 集成学习

定义: 通过结合多个学习器(例如同种算法但是参数不同, 或者不同算法), 一般会获得比任意单个学习器都要好的性能, 尤其是在这些学习器都是"弱学习器"的时候提升效果会很明显。

[调参学习链接](#)

## 10.1 Boosting(提升法)

可以用于回归和分类问题，它每一步产生一个弱预测模型（如**决策树**），并加权累加到总模型中；如果每一步的弱预测模型生成都是依据损失函数的梯度方向，则称之为梯度提升。

梯度提升算法首先给定一个目标损失函数，它的定义域是所有可行的弱函数集合，提升算法通过迭代的选择一个负梯度方向上的基函数来逐渐逼近局部最小值。

提升的**理论意义**：如果一个问题存在弱分类器，则可以通过提升的办法得到强分类器。

### 10.1.1 梯度提升(GBDT)

**DT** 表示使用决策树作为基学习器，使用的 **CART** 树。

GBDT 是迭代，但 GBDT 每一次的计算都是为了**减少上一次的残差**，进而在残差减少（**负梯度**）的方向上建立一个新的模型，其弱学习器限定了只能使用 CART 回归树模型。残差=（实际值-预测值）



#### 10.1.1.1 GBDT 是训练过程如何选择特征？

GBDT 使用基学习器是 **CART** 树，CART 树是二叉树，每次使用 yes or no 进行特征选择，数值连续特征使用的最小均方误差，离散值使用的 gini 指数。在每次划分特征的时候会遍历所有可能的划分点找到最有特征分裂点，这是为什么 gbdt 会比 rf 慢的主要原因之一。

#### 10.1.1.2 GBDT 如何防止过拟合？由于 gbdt 是前向加法模型，前面的树往往起到决定性的作用，如何改进这个问题？

一般使用**缩减因子**对每棵树进行降权，可以使用带有 **dropout** 的 GBDT 算法，dart 树，随机丢弃生成的决策树，然后再从剩下的决策树集中迭代优化提升树。

GBDT 与 Boosting 区别较大，它的每一次计算都是为了**减少上一次的残差**，而为了消除残差，可以在残差减小的梯度方向上建立模型；

在 GradientBoost 中，每个新的模型的建立是为了使得之前的模型的残差往梯度下降的方法。

#### 10.1.1.3 梯度提升的如何调参？

1.首先我们从步长(**learning rate**)和迭代次数(**n\_estimators**)入手。

开始选择一个较小的步长来网格搜索最好的迭代次数。将**步长**初始值设置为 0.1；

```
param_test1 = {'n_estimators':range(20,81,10)}
gsearch1 = GridSearchCV(estimator = GradientBoostingClassifier(
    learning_rate=0.1, min_samples_split=300,
    min_samples_leaf=20,max_depth=8,max_features='sqrt', subsample=0.8,random_state=10),
    param_grid = param_test1, scoring='roc_auc',iid=False,cv=5)
gsearch1.fit(X,y)
gsearch1.grid_scores_, gsearch1.best_params_, gsearch1.best_score_
```

2.找到了一个合适的迭代次数，对决策树进行调参。首先对决策树最大深度 **max\_depth** 和内部节点再划分所需最小样本数(**min\_samples\_split**)进行网格搜索。

```
param_test2 = {'max_depth':range(3,14,2), 'min_samples_split':range(100,801,200)}
gsearch2 = GridSearchCV(estimator = GradientBoostingClassifier(
    learning_rate=0.1, n_estimators=60, min_samples_leaf=20,
    max_features='sqrt', subsample=0.8, random_state=10),
    param_grid = param_test2, scoring='roc_auc',iid=False, cv=5)
```

再对 **min\_samples\_split** 和叶子节点最少样本数(**min\_samples\_leaf**)一起调参。

```
param_test3 = {'min_samples_split':range(800,1900,200),
    'min_samples_leaf':range(60,101,10)}
```

得出：{'min\_samples\_leaf': 60, 'min\_samples\_split': 1200},

3.对比最开始 完全不调参 的拟合效果，可见精确度稍有下降，主要原理是我们使用了 0.8 的子采样，20%的数据没有参与拟合。

需要再对最大特征数(**max\_features**)进行网格搜索。

|                   |                                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------|
| learning_rate     | 小的学习率需要更多的树(n_estimators);建议lr<=0.1                                                                                       |
| n_estimators      | 最多训练多少棵树                                                                                                                  |
| max_depth         | 每棵子树的 <b>深度</b> ，默认为3;如果数据量和特征都不多，可以不管这个参数。但是当较大时，建议限制深度，10-100之间。max_depth= <b>k</b> 和 max_leaf_nodes= <b>k-1</b> 的效果差不多 |
| max_leaf_nodes    | <b>最大叶子节点数量</b> ，默认为None,在限制的叶节点数之内生成最优决策树，可以防止过拟合。当数量级较大，可以限制这个数。                                                        |
| max_features      | 划分时考虑的 <b>特征数量</b> ；当特征数量并不多(<50)，为None;列取样                                                                               |
| subsample         | <b>子采样比例</b> ，默认1.0，建议0.5~0.8，是不放回的采样；行取样                                                                                 |
| min_samples_leaf  | <b>叶子节点最少的样本数</b> ，默认1。                                                                                                   |
| min_samples_split | 子树继续划分的条件，默认为2；当一个节点内的样本数量少于该值时，该节点不再拆分，当作叶节点。数据量小不用管，数据量大可以增大该值。                                                         |

#### 10.1.1.4 GBDT 对标量特征要不要 one-hot 编码？

从效果的角度来讲，使用 category 特征和 one-hot 是等价的，所不同的是 category 特征的 feature 空间更小。微软在 lightGBM 的文档里也说了，category 特征可以直接输入，不需要 one-hot 编码，准确度差不多，速度快 8 倍。而 sklearn 的 tree 方法在接口上不支持 category 输入，所以只能用 one-hot 编码。

#### 10.1.1.5 为什么 GBDT 用负梯度当做残差？

我们希望找到一个  $f(x)$  使得  $L(y, f(x))$  最小，那么  $f(x)$  就得沿着使损失函数  $L$  减小的方向变化，即：

$$f(x_1) = f(x) - \frac{\partial L(y, f(x))}{\partial f(x)}$$

同时，最新的学习器是由当前学习器  $f(x)$  与本次要产生的回归树  $T_1$  相加得到的：

$$f(x_1) = f(x) + T_1$$

因此，为了让损失函数减小，需要令：

$$-\frac{\partial L(y, f(x))}{\partial f(x)} = T_1$$

1. 负梯度的方向可证，模型优化下去一定会收敛
2. 对于一些损失函数来说最大的残差方向，并不是梯度下降最好的方向，倒是损失函数最小与残差最小两者目标不统一

### 10.1.2 自适应提升(AdaBoost)

定义：是一种提升方法，将多个弱分类器，组合成强分类器。

Adaboost 既可以用作分类，也可以用作回归。

算法实现：

1. 提高上一轮被错误分类的样本的权值，降低被正确分类的样本的权值；
2. 线性加权求和。误差率小的基学习器拥有较大的权值，误差率大的基学习器拥有较小的权值。

| 优点                        | 实际应用              |
|---------------------------|-------------------|
| (1) 精度很高的分类器              | (1) 用于二分类或多分类     |
| (2) 提供的是框架，可以使用各种方法构建弱分类器 | (2) 特征选择          |
| (3) 简单，不需要做特征筛选           | (3) 分类人物的baseline |
| (4) 不用担心过度拟合              |                   |

#### 10.1.2.1 为什么 Adaboost 方式能够提高整体模型的学习精度？

根据前向分布加法模型，Adaboost 算法每一次都会降低整体的误差，虽然单个模型误差会有波动，但是整体的误差却在降低，整体模型复杂度在提高。

#### 10.1.2.2 使用 m 个基学习器和加权平均使用 m 个学习器之间有什么不同？

Adaboost 的 m 个基学习器是有顺序关系的，第 k 个基学习器根据前 k-1 个学习器得到的误差更新数据分布，再进行学习，每一次的数据分布都不同，是使用同一个学习器在不同的数据分布上进行学习。

加权平均的 m 个学习器是可以并行处理的，在同一个数据分布上，学习得到 m 个不同的学习器进行加权。

#### 10.1.2.3 adaboost 的迭代次数(基学习器的个数)如何控制？

一般使用 **earlystopping** 进行控制迭代次数。

#### 10.1.2.4 adaboost 算法中基学习器是否很重要，应该怎么选择基学习器？

sklearn 中的 adaboost 接口给出的是使用决策树作为基分类器，一般认为决策树表现良好，其实可以根据数据的分布选择对应的分类器，比如选择简单的 LR，或者对于回归问题选择线性回归。

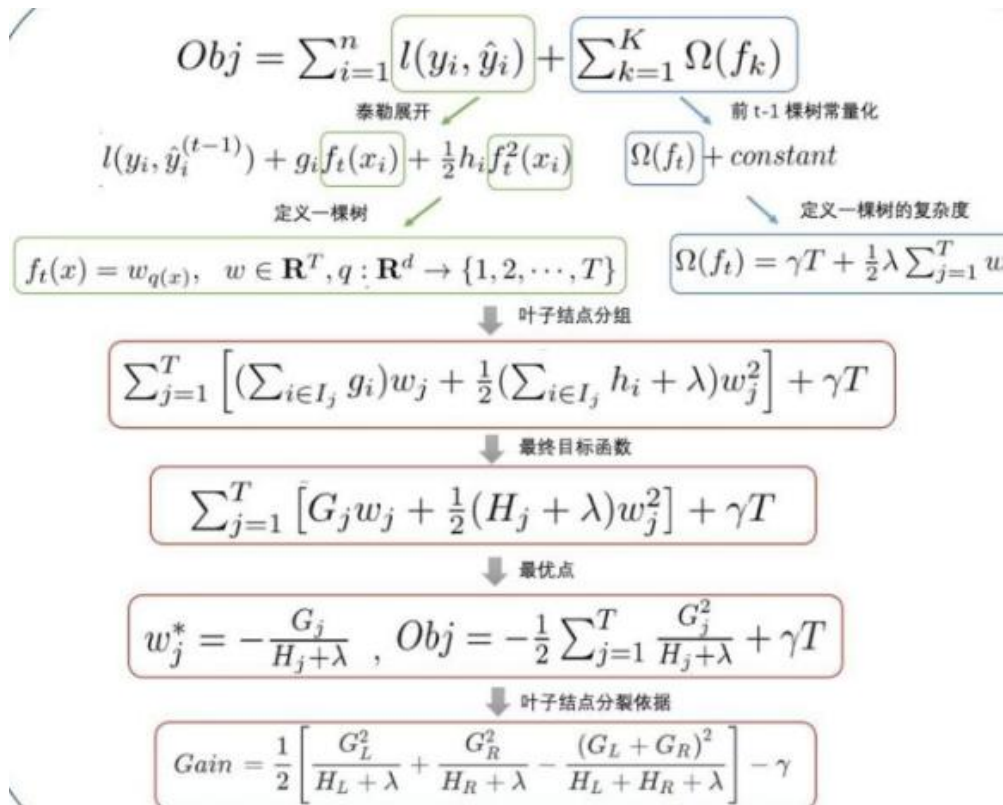
### 10.1.3 极端梯度提升(XGBoost)

基于 boosting **增强**策略的加法模型，训练的时候采用**前向分布**算法进行贪婪

的学习，每次迭代都学习一棵 **CART** 树来拟合之前 t-1 棵树的预测结果与训练样本真实值的残差。

XGBoost 对 GBDT 进行了一系列优化，比如损失函数进行了**二阶泰勒展开**、目标函数加入**正则项**、支持**并行**和默认**缺失值处理**等，在可扩展性和训练速度上有了巨大的提升，但其核心思想没有大的变化。

### XGBoost - 推导指示图



XGBoost 的学习分为 3 步：

① 集成思想 ② 损失函数分析 ③ 求解

#### 10.1.3.1 XGBoost 使用泰勒二阶展开的原因？

**精准性：**相对于 GBDT 的一阶泰勒展开，XGBoost 采用二阶泰勒展开，可以更为精准的逼近真实的损失函数

**可扩展性：**损失函数支持自定义，只需要新的损失函数二阶可导。

#### 10.1.3.2 XGBoost 可以并行训练的原因？

XGBoost 的并行，并不是说每棵树可以并行训练，XGB 本质上仍然采用 boosting 思想，每棵树训练前需要等前面的树训练完成才能开始训练。

XGBoost 的并行，指的是特征维度的并行：在训练之前，每个特征按特征值对样本进行预排序，并存储为 Block 结构，在后面查找特征分割点时可以重复使用，而且特征已经被存储为一个个 block 结构，那么在寻找每个特征的最佳分割点时，可以利用多线程对每个 block 并行计算。

#### 10.1.3.3 XGBoost 为什么快？

|                |                                                               |
|----------------|---------------------------------------------------------------|
| 分块并行:          | 训练前每个特征按特征值进行排序并存储为Block结构, 后面查找特征分割点时重复使用, 并且支持并行查找每个特征的分割点  |
| 候选分位点:         | 每个特征采用常数个分位点作为候选分割点                                           |
| CPU cache命中优化: | 使用缓存预取的方法, 对每个线程分配一个连续的buffer, 读取每个block中样本的梯度信息并存入连续的Buffer中 |
| Block 处理优化:    | Block预先放入内存;                                                  |
|                | Block按列进行解压缩;                                                 |
|                | 将Block划分到不同硬盘来提高吞吐                                            |

#### 10.1.3.4 XGBoost 防止过拟合的方法?

|            |                               |
|------------|-------------------------------|
| 目标函数添加正则项: | 叶子节点个数+叶子节点权重的L2正则化           |
| 列抽样:       | 训练的时候只用一部分特征 (不考虑剩余的block块即可) |
| 子采样:       | 每轮计算可以不使用全部样本, 使算法更加保守        |
| shrinkage: | 可以叫学习率或步长, 为了给后面的训练留出更多的学习空间  |

#### 10.1.3.5 XGBoost 如何处理缺失值?

在特征 k 上寻找最佳 split point 时, 不会对该列特征 missing 的样本进行遍历, 而只对该列特征值为 non-missing 的样本上对应的特征值进行遍历, 通过这个技巧来减少了为稀疏离散特征寻找 split point 的时间开销。

在逻辑实现上, 为了保证完备性, 会将该特征值 missing 的样本分别分配到左叶子节点和右叶子节点, 两种情形都计算一遍后, 选择分裂后增益最大的那个方向 (左分支或是右分支), 作为预测时特征值缺失样本的默认分支方向。

如果在训练中没有缺失值而在预测中出现缺失, 那么会自动将缺失值的划分方向放到右子节点。

#### 10.1.3.6 为什么 XGBoost 相比某些模型对缺失值不敏感?

对存在缺失值的特征, 一般的解决方法是:

离散型变量: 用出现次数最多的特征值填充;

连续型变量: 用中位数或均值填充;

一棵树中每个结点在分裂时, 寻找的是某个特征的最佳分裂点 (特征值), 完全可以不考虑存在特征值缺失的样本,

也就是说, 如果某些样本缺失的特征值缺失, 对寻找最佳分割点的影响不是很大。

对于有缺失值的数据在经过缺失处理后:

当数据量很小时, 优先用朴素贝叶斯;

数据量适中或者较大, 用树模型, 优先 XGBoost;

数据量较大, 也可以用神经网络;

避免使用距离度量相关的模型, 如 KNN 和 SVM。

#### 10.1.3.7 XGBoost 如何处理不平衡数据?

对于不平衡的数据集, 例如用户的购买行为, 肯定是极其不平衡的, 这对 XGBoost 的训练有很大的影响, XGBoost 有两种自带的方法来解决:

第一种, 如果你在意 AUC, 采用 AUC 来评估模型的性能, 那你可以通过设置 scale\_pos\_weight 来平衡正样本和负样本的权重。例如, 当正负样本比例为 1:10 时, scale\_pos\_weight 可以取 10;

第二种，如果你在意**概率**(预测得分的合理性)，你不能重新平衡数据集(会破坏数据的真实分布)，应该设置 `max_delta_step` 为一个有限数字来帮助收敛（基模型为 LR 时有效）。

### 10.1.3.8 XGBoost 中叶子结点的权重如何计算出来？

XGBoost 目标函数最终推导形式如下：

$$Obj^{(t)} = \sum_{j=1}^T [G_j w_j + \frac{1}{2}(H_j + \lambda)w_j^2] + \gamma T$$

利用一元二次函数求最值的知识，当目标函数达到最小值  $Obj^*$  时，每个叶子结点的权重为  $w_j^*$ 。

具体公式如下：

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad Obj = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

↑ 每个叶子结点的权重 (score)
 ↑ 第 t 棵树带来的最小损失(训练损失+正则化损失)

### 10.1.3.9 XGBoost 中的一棵树的停止生长条件？

当新引入的一次分裂所带来的增益  $Gain < 0$  时，放弃当前的分裂。这是训练损失和模型结构复杂度的博弈过程。

当树达到最大深度时，停止建树，因为树的深度太深容易出现过拟合，这里需要设置一个超参数 `max_depth`。

当引入一次分裂后，重新计算新生成的左、右两个叶子结点的样本权重和。如果任一个叶子结点的样本权重低于某一个阈值，也会放弃此次分裂。这涉及到一个超参数：最小样本权重和，是指如果一个叶子节点包含的样本数量太少也会放弃分裂，防止树分的太细。

### 10.1.3.10 比较 LR 和 GBDT，说些什么情景下 GBDT 不如 LR？

LR 是线性模型，可解释性强，很容易并行化，但学习能力有限，需要大量的人工特征工程；

GBDT 是非线性模型，具有天然的特征组合优势，特征表达能力强，但是树与树之间无法并行训练，而且树模型很容易过拟合；

当在高维稀疏特征的场景下，LR 的效果一般会比 GBDT 好。原因如下：

先看一个例子：

假设一个二分类问题，label 为 0 和 1，特征有 100 维，如果有 1w 个样本，但其中只要 10 个正样本 1，而这些样本的特征 `f1` 的值为全为 1，而其余 9990 条样本的 `f1` 特征都为 0(在高维稀疏的情况下这种情况很常见)。

树模型很容易优化出一个使用 `f1` 特征作为重要分裂节点的树，因为这个结点直接能够将训练数据划分的很好，但是当测试的时候，却会发现效果很差，因为这个特征 `f1` 只是刚好偶然间跟 `y` 拟合到了这个规律，这也是我们常说的过拟合。

那么这种情况下，如果采用 LR 的话，应该也会出现类似过拟合的情况呀： $y = W_1 * f_1 + W_i * f_i + \dots$ ，其中  $W_1$  特别大以拟合这 10 个样本。

为什么此时树模型就过拟合的更严重呢？

因为现在的模型普遍都会带着正则项，而 LR 等线性模型的正则项是对权重的惩罚，也就是  $W_1$  一旦过大，惩罚就会很大，进一步压缩  $W_1$  的值，使他不至于过大。但是，树模型则不一样，树模型的惩罚项通常为叶子节点数和深度等，而我们都知，对于上面这种 case，树只需要一个节点就可以完美分割 9990 和 10 个样本，一个结点，最终产生的惩罚项极其之小。

这也就是为什么在高维稀疏特征的时候，线性模型会比非线性模型好的原因了：带正则化的线性模型比较不容易对稀疏特征过拟合。

#### 10.1.3.11 XGBoost 在什么地方做的剪枝？如何进行剪枝？

- (1) 目标函数时，使用叶子的数目和  $l_2$  模的平方，控制模型的复杂度
  - (2) 在分裂节点的计算增益中，定义了一个阈值，当增益大于阈值才分裂
- 先从顶到底建立树直到最大深度，再从底到顶反向检查是否有不满足分裂条件的结点，进行剪枝。

#### 10.1.3.12 XGBoost 如何选择最佳分裂点？

XGBoost 在训练前预先将特征按照特征值进行了排序，并存储为 **block** 结构，以后在结点分裂时可以重复使用该结构。

因此，可以采用**特征并行**的方法利用多个线程分别计算每个特征的最佳分割点，根据每次分裂后产生的增益，最终选择增益最大的那个特征的特征值作为最佳分裂点。如果在计算每个特征的最佳分割点时，对每个样本都进行遍历，计算复杂度会很大，这种全局扫描的方法并不适用大数据的场景。XGBoost 还提供了一种**直方图近似算法**，对特征排序后仅选择常数个候选分裂位置作为候选分裂点，极大提升了结点分裂时的计算效率。

#### 10.1.3.13 XGBoost 的 Scalable 性如何体现？

**基分类器**的 scalability: 弱分类器可以支持 CART 决策树，也可以支持 LR 和 Linear。

**目标函数**的 scalability: 支持自定义 loss function，只需要其一阶、二阶可导。有这个特性是因为泰勒二阶展开，得到通用的目标函数形式。

**学习方法**的 scalability: Block 结构支持并行化，支持 Out-of-core 计算。

#### 10.1.3.14 XGBoost 参数调优的一般步骤？

首先需要初始化一些基本变量，例如：

```
max_depth = 5
min_child_weight = 1
gamma = 0
subsample, colsample_bytree = 0.8
scale_pos_weight = 1
```

- (1) 确定 **learning rate** 和 **estimator** 的数量 learning rate 可以先用 0.1，用 cv 来寻找最优的 estimators(2) max\_depth 和 min\_child\_weight

我们调整这两个参数是因为，这两个参数对输出结果的影响很大。我们首先将这两个参数设置为较大的数，然后通过迭代的方式不断修正，缩小范围。

**max\_depth**，每棵子树的最大深度，check from range(3,10,2)。

**min\_child\_weight**，子节点的权重阈值，check from range(1,6,2)。

如果一个结点分裂后，它的所有子节点的权重之和都大于该阈值，该叶子节点才可以划分。

- (3) **gamma** 也称作最小划分损失 min\_split\_loss，check from 0.1 to 0.5，指的是，对于一个叶子节点，当对它采取划分之后，损失函数的降低值的阈值。

如果大于该阈值，则该叶子节点值得继续划分；

如果小于该阈值，则该叶子节点不值得继续划分。

- (4) **subsample, colsample\_bytree**

subsample 是对训练的采样比例

colsample\_bytree 是对特征的采样比例

both check from 0.6 to 0.9

(5) 正则化参数

alpha 是 L1 正则化系数, try 1e-5, 1e-2, 0.1, 1, 100

lambda 是 L2 正则化系数

(6) 降低学习率降低学习率的同时增加树的数量, 通常最后设置学习率为 0.01~0.1

**10.1.3.15 XGBoost 模型如果过拟合了怎么解决?**

当出现过拟合时, 有两类参数可以缓解:

**第一类参数:** 用于直接控制模型的复杂度。

包括 max\_depth,min\_child\_weight,gamma 等参数

**第二类参数:** 用于增加随机性, 从而使得模型在训练时对于噪音不敏感。

包括 subsample,colsample\_bytree 还有就是直接减小 learning rate, 但需要同时增加 **estimator** 参数。

**10.1.3.16 XGBoost 如何寻找最优特征? 是有放回还是无放回?**

XGBoost 利用梯度优化模型算法, 样本是不放回的。但 XGBoost 支持子采样, 也就是每轮计算可以不使用全部样本。

**10.1.3.17 XGBoost 如何分布式? 特征分布式和数据分布式? 各有什么问题?**

XGBoost 在训练之前, 预先对数据按列进行排序, 然后保存 block 结构。

(1) 特征分布式(特征间并行): 由于将数据按列存储, 可以同时访问所有列, 那么可以对所有属性同时执行切分点寻找算法, 从而并行化切分点寻找;

(2) 数据分布式(特征内并行): 可以用多个 block 分别存储不同的样本集, 多个 block 可以并行计算。

问题: (1) 不能从本质上减少计算量; (2) 通讯代价高。

**10.1.3.18 为什么 XGBoost 的近似算法比 lightgbm 慢很多呢?**

xgboost 在每一层都动态构建直方图, 因为 xgboost 的直方图算法不是针对某个特定的 feature, 而是所有 feature 共享一个直方图(每个样本的权重是二阶导), 所以每一层都要重新构建直方图, 而 lightgbm 中对每个特征都有一个直方图, 所以构建一次直方图就够了。

**10.1.4 LightGBM**

基本思想: 先把连续的浮点特征值离散化成 **k 个整数**, 同时构造一个宽度为 **k** 的直方图。在遍历数据的时候, 根据离散化后的值作为索引在直方图中累积统计量, 当遍历一次数据后, 直方图累积了需要的统计量, 然后根据直方图的离散值, 遍历寻找最优的分割点;

基于**决策树**算法的**分布式梯度提升**框架; 是对 GBDT 的高效实现, 原理上它和 GBDT 及 XGBoost 类似, 都采用损失函数的负梯度作为当前决策树的残差近似值, 去拟合新的决策树。

|    |                        | 解释                                     |
|----|------------------------|----------------------------------------|
| 优点 | Histogram算法            | 先把连续的浮点特征值离散化成k个整数，同时构造一个宽度为k的直方图      |
|    | 带深度限制的Leaf-wise的叶子生长策略 | 树是按层生长的，同一层的所有节点都做分裂，最后剪枝。             |
|    | 直方图差加速                 |                                        |
|    | 直接支持类别特征               |                                        |
|    | 速度较快                   | 是XGBoost速度的16倍，内存占用率为XGBoost的1/6       |
| 缺点 | 可能会长出比较深的决策树，产生过拟合     | 在Leaf-wise之上增加了一个最大深度限制，在保证高效率的同时防止过拟合 |
|    | 基于偏差的算法，会对噪点较为敏感       |                                        |

### 10.1.5 CatBoost

原理：**One-hot** 编码可以在预处理阶段或在训练期间完成；处理类别型特征棒。

具体实现方法如下：

- 1.将输入样本集**随机排序**，并生成多组随机排列的情况；
- 2.将浮点型或属性值标记转化为整数；
- 3.将所有的分类特征值结果都根据以下公式，转化为数值结果。

$$\frac{\sum_{j=1}^{p-1} [x_{\sigma_j, k} = x_{\sigma_p, k}] Y_{\sigma_j} + a \cdot P}{\sum_{j=1}^{p-1} [x_{\sigma_j, k} = x_{\sigma_p, k}] + a}$$

## 10.2 Bagging(套袋法)

算法过程如下：

从原始样本集中使 **Bootsrapping** 方法随机抽取 n 个训练样本,共进行轮抽取，得到 k 个训练集。（k 个训练集之间相互独立，元素可以有重复）

对于 k 个训练集，我们训练 k 个模型(这 k 个模型可以根据具体问题而定,比如决策树, knn 等)

对于**分类**问题:由**投票**表决产生分类结果;

对于**回归**问题: 由 k 个模型预测结果的**均值**作为最后预测结果。

**Bagging + 决策树 = 随机森林**

**AdaBoost + 决策树 = 提升树**

**Gradient Boosting + 决策树 = GBDT**

### 10.2.1 随机森林

定义: 随机森林就是通过集成学习的思想将多棵树集成的一种算法，它的基本单元是**决策树**，而它的本质属于集成学习方法。它的工作原理是生成多个分类器/模型，各自独立地学习和作出预测。这些预测最后结合成单预测，因此优于任何一个单分类的做出预测。

### 算法思想:

1.随机选择样本（放回抽样）--> 2.随机选择特征 --> 3.构建决策树 --> 4.随机森林投票（平均）

| 优点                                                                                             | 缺点                                                                                        |
|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 1. 并行                                                                                          | 1. 在解决回归问题时，表现较差，这是因为它并不能给出一个连续的输出；                                                       |
| 2. 随机性的引入，增加了多样性，泛化能力非常强，抗噪声能力强，对缺失值不敏感；                                                       | 2. 在某些噪音较大的分类或者回归问题上会过拟合；                                                                 |
| 3. 可省略交叉验证，因为随机采样；                                                                             | 3. 对于许多统计建模者来说，无法控制模型内部运行（ <b>可控性差</b> ）；                                                 |
| 4. 继承决策树有的优点，包括：<br>（1）可得到特征重要性排序，因此可做“特征选择”；<br>（2）可处理高维特征，且不用特征选择；<br>（3）能处理离散型/连续型数据，无需规范化； | 4. 对于特征较少的数据，可能不能产生很好的分类；<br>5. 可能有很多相似的决策树，掩盖了真实的结果；<br>6. 执行速度虽然比boosting等快，但比单只决策树慢多了。 |

#### 10.2.1.1 随机森林的随机性指的是？

- 1.决策树训练样本是有放回随机采样的；
- 2.决策树节点分裂特征集是有放回随机采样的；

#### 10.2.1.2 为什么随机抽样？

保证基分类器的多样性，若每棵树的样本集都一样，训练的每棵决策树都是一样

#### 10.2.1.3 为什么要有放回的抽样？

保证样本集间有重叠，若不放回，每个训练样本集及其分布都不一样，可能导致训练的各决策树差异性很大，最终多数表决无法“求同”，即最终多数表决相当于“求同”过程。

#### 10.2.1.4 为什么不用全样本训练？

全样本忽视了局部样本的规律，不利于模型泛化能力

#### 10.2.1.5 为什么要随机特征？

随机特征保证基分类器的多样性（差异性），最终集成的泛化性能可通过个体学习器之间的差异度而进一步提升，从而提高泛化能力和抗噪能力

#### 10.2.1.6 需要剪枝吗？

不需要，后剪枝是为了避免过拟合，随机森林随机选择变量与树的数量，已经避免了过拟合，没必要去剪枝了。一般 rf 要控制的是树的规模，而不是树的置信度，剩下的每棵树需要做的就是尽可能的在自己所对应的数据(特征)集情况下尽可能的做到最好的预测结果。剪枝的作用其实被集成方法消解了，所以用处不大

#### 10.2.1.7 随机森林如何处理缺失值？

- 1.对于训练集,同一个类下的数据：如果是分类变量缺失，用**众数**补上；如果是连续型变量缺失，用**中位数**补。
- 2.先用方法 1 补上缺失值，然后构建森林并计算**相似矩阵**，再回头看缺失值，如

果是分类变量，则用没有阵进行**加权平均**的方法补缺失值。然后迭代 4-6 次。

#### 10.2.1.8 随机森林如何评估特征重要性？

1.Decrease GINI: 对于回归问题，直接使用  $\text{argmax}$  作为评判标准，即当前节点训练集的方差(Var)减去左节点的方差(VarLeft)和右节点的方差(VarRight)。

2.Decrease Accuracy: 对于一棵树，我们用 OOB 样本可以得到测试误差 1；然后随机改变 OOB 样本的第  $j$  列：保持其他列不变，对第  $j$  列进行随机的上下置换，得到误差 2。可以用(误差 1-误差 2)来刻画变量  $j$  的重要性。

**基本思想:** 如果一个变量  $j$  足够重要，那么改变它会极大的增加测试误差；反之，如果改变它测试误差没有增大，则说明该变量不是那么的重要。

#### 10.2.1.9 RF 与决策树的区别？

- (1) RF 是决策树的集成；
- (2) RF 中是“随机属性型”决策树

#### 10.2.1.10 RF 为什么比 bagging 效率高？

因为在个体决策树的构建过程中，Bagging 使用的是“确定型”决策树，bagging 在选择划分属性时要对每棵树是对所有特征进行考察；而随机森林仅仅考虑一个特征子集。

#### 10.2.1.11 RF 为什么能够更鲁棒？

由于 RF 使用了使用了**行采样**和**列采样**技术，是的每棵树不容易过拟合；并且是基于树的集成算法，由于使用了采用数据是的每棵树的差别较大，在进行 embedding 的时候可以更好的降低模型的方差，整体而言是的 RF 是一个鲁棒的模型。

#### 10.2.1.12 RF 分类和回归问题如何预测 $y$ 值？

RF 是一个**加权平均**的模型，是进行分类问题的时候，使用的个  $k$  个树的投票策略，**多数服从少数**。在回归的使用是使用的  $k$  个树的平均。可以看出来 rf 的训练和预测过程都可以进行并行处理。

#### 10.2.1.13 为什么 RF 的树比 GBDT 的要深一点？

RF 是通过**投票**的方式来降低方差，但是本质上需要每棵树有较强的表达能力，所以单颗树深点没关系，通过投票的方式降低过拟合。而 GBDT 是通过加强前一棵树的表达能力，所以每颗树不必有太强的表达能力。可以通过 **boosting** 的方式来提高，也能提高训练速度（gbdt 害怕过拟合，rf 不怕，通过投票的方式杜绝）

### 10.3 Bagging vs Boosting

bagging (S 个数据集，并行训练，投票法)

boosting (一个数据集，串行训练，加权求和)

|                   | Bagging                                      | Boosting                                                             |
|-------------------|----------------------------------------------|----------------------------------------------------------------------|
| 从 <b>样本选择</b> 角度  | 采用随机有放回的采样方式(Bootstrap)                      | 使用所有样本，但每个样本的权重不同                                                    |
| 从 <b>决策方式</b> 角度  | <b>投票选举法</b> ，回归预测采用各基分类器预测结果的 <b>平均值</b>    | 各基分类器在不同权重作用下预测结果的 <b>累加和</b>                                        |
| 从 <b>方差、偏差</b> 角度 | 以随机采样样本的方式减少异常样本的选择比例，从而可以降低过拟合， <b>减小方差</b> | 损失函数就是以 <b>减少偏差</b> 为目的来训练下一个基分类器                                    |
| 从 <b>权重</b> 角度    | 各个样本的权重相同，各个基分类器权重相同                         | 各个样本的权重不同，正确预测的样本权重减小，错误预测的样本权重增大；各个基分类器的权重不同，预测准确率高的权重大，预测准确率低的权重小； |

#### 10.4 随机森林 vs GBDT

|              | 随机森林                                 | GBDT                  |
|--------------|--------------------------------------|-----------------------|
| <b>并行和串行</b> | 并行算法                                 | 串行算法                  |
| <b>决策方式</b>  | 采用大多数投票选举法，回归问题采用各基分类器结果的 <b>平均值</b> | 各基分类器预测结果的 <b>累加和</b> |
| <b>样本选择</b>  | 采用有放回随机采样的方式                         | 使用所有的样本               |
| <b>偏差、方差</b> | 降低 <b>方差</b> 提高性能                    | 降低 <b>偏差</b> 提高性能     |
| <b>异常值</b>   | 不敏感                                  | 敏感                    |

#### 10.5 AdaBoost vs GBDT

|    | AdaBoost                    | GBDT                       |
|----|-----------------------------|----------------------------|
| 联系 | 通过降低 <b>偏差</b> 提高模型精度       |                            |
|    | 都是 <b>前项分布加法</b> 模型的一种      |                            |
| 区别 | 通过不断修改权重、不断加入弱分类器进行boosting | 通过不断在负梯度方向上加入新的树进行boosting |

#### 10.6 XGBoost vs GBDT

|    |            | XGBoost                                                                                            | GBDT                                                         |
|----|------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
|    | 基分类器       | 不仅支持CART决策树，还支持 <b>线性分类器</b>                                                                       | 仅支持CART                                                      |
| 区别 | 正则项        | 损失函数添加了 <b>正则化</b> 项，使用正则用以控制模型的复杂度，正则项里包含了树的叶子节点数( $\gamma$ )、节点权重和( $\min\_child\_weight$ )。     | 仅通过 <b>学习率</b> 来做一个正则化，此外 <b>early stopping</b> 也达到了一个正则化的效果 |
|    | 特征重要性的判断标准 | 三种方法：<br><b>weight</b> :特征用来作为切分特征的次数<br><b>gain</b> :使用特征进行切分的平均增益<br><b>cover</b> :各个树中该特征平均覆盖情况 | 根据树的节点特征对应的 <b>深度</b> 来判断                                    |
|    | 导数信息       | 使用了一、 <b>二阶</b> 导数信息                                                                               | 只使用了一阶导数信息                                                   |
|    | 缺失值处理      | 对树中的每个非叶子节点，XGBoost可以自动学习出它的默认分裂方向。如果某个样本该特征值缺失，会将其划入默认分支。                                         |                                                              |
|    | <b>列抽样</b> | 支持列采样，与随机森林类似，用于 <b>防止过拟合</b>                                                                      |                                                              |
|    | 并行化        | <b>特征维度的并行</b> 。预先将每个特征按特征值排好序，存储为块结构，分裂结点时可以采用多线程并行查找每个特征的最佳分割点，极大提升训练速度。                         |                                                              |

## 10.7 XGBoost vs LightGBM

|    |                       | XGBoost               | LightGBM                 |
|----|-----------------------|-----------------------|--------------------------|
| 区别 | 树的切分策略                | <b>level-wise</b>     | <b>leaf-wise</b>         |
|    | 实现并行的方式               | 预排序的方式                | <b>直方图算法</b>             |
|    | 多项评价指标同时评价时两者的早停止策略不同 | 根据评价指标列表中的最后一项来作为停止标准 | 受到所有评价指标的影响              |
|    |                       |                       | 直接支持类别特征，对类别特征不必进行独热编码处理 |

## 11.无监督学习

### 11.1 聚类

**原理:** 对大量未知标注的数据集，按数据的内在相似性将数据集划分为多个类别，使类别内的数据相似度较大而类别间的数据相似度较小。

**聚类的应用场景:** **求职信息完善**（有大约 10 万份优质简历，其中部分简历包含完整的字段，部分简历在学历，公司规模，薪水，等字段有些置空项。希望对数据进行学习，编码与测试，挖掘出职位路径的走向与规律，形成算法模型，在对数据中置空的信息进行预测。）

|    | 算法         | 英文                           |
|----|------------|------------------------------|
| 聚类 | <b>K均值</b> | K-means                      |
|    | 学习向量量化     | Learning Vector Quantization |
|    | 高斯混合聚类     | Mixture-of-Gaussian          |
|    | 密度聚类       | DBSCAN                       |
|    | 层次聚类       | AGNES                        |

### 11.1.1 K-means

**定义：**也叫 K 均值或 K 平均。通过**迭代**的方式，每次迭代都将数据集中的各个点划分到距离它最近的簇内，这里的距离即数据点到簇中心的距离。

**K-means 步骤：**

- 1.随机初始化 K 个簇中心坐标
- 2.计算数据集内所有点到 K 个簇中心的距离，并将数据点划分近最近的簇
- 3.更新簇中心坐标为当前簇内节点的坐标平均值
- 4.重复 2、3 步骤直到簇中心坐标不再改变（收敛了）

#### 11.1.1.1 K 值的如何选取？

K-means 算法要求事先知道数据集能分为几群，主要有两种方法定义 K。

**elbow method** 通过绘制 K 和损失函数的关系图，选拐点处的 K 值。

经验选取人工据经验先定几个 K，多次随机初始化中心选经验上最适合的。

通常都是以经验选取，因为实际操作中拐点不明显，且 elbow method 效率不高。

#### 11.1.1.2 K-means 算法中初始点的选择对最终结果的影响？

K-means 选择的初始点不同获得的最终分类结果也可能不同，随机选择的中心会导致 K-means 陷入局部最优解。

#### 11.1.1.3 K-means 不适用哪些数据？

- 1.数据特征极强相关的数据集，因为会很难收敛（损失函数是非凸函数），一般要用 Kernal K-means，将数据点映射到更高维度再分群。
- 2.数据集可分出来的簇密度不一，或有很多离群值（outliers），这时候考虑使用密度聚类。

#### 11.1.1.4 K-means 中常用的距离度量？

K-means 中比较常用的距离度量是欧几里得**距离**和余弦相似度。

K-means 是否会一直陷入选择质心的循环停不下来（为什么迭代次数后会收敛）？

从 K-means 的第三步我们可以看出，每回迭代都会用簇内点的平均值去更新簇中心，所以最终簇内的平方误差和（SSE, sum of squared error）一定最小。平方误差和的公式如下：

$$L(X) = \sum_{i=1}^K \left\{ \sum_{j \in C_i} (x_{ij} - \bar{x}_i)^2 \right\}$$

#### 11.1.1.5 为什么在计算 K-means 之前要将数据点在各维度上归一化？

因为数据点各维度的量级不同，例如：最近正好做完基于 RFM 模型的会员分群，每个会员分别有 R（最近一次购买距今的时长）、

F（来店消费的频率）和 M（购买金额）。如果这是一家奢侈品商店，你会发现 M 的量级（可能几万元）远大于 F（可能平均 10 次以下），如果不归一化就算 K-means，相当于 F 这个特征完全无效。如果我希望能把常客与其他顾客区别开来，不归一化就做不到。

### 11.1.1.6 聚类 and 分类区别？

1. 产生的结果相同（将数据进行分类）
2. 聚类事先没有给出标签（无监督学习）

最大的不同在于：分类的目标是事先已知的，而聚类则不一样，聚类事先不知道目标变量是什么，类别没有像分类那样被预先定义出来。

### 11.1.2 K-means vs KNN

|     |      | KNN                                                                       | K-Means                                       |
|-----|------|---------------------------------------------------------------------------|-----------------------------------------------|
| 区别  |      | 分类算法                                                                      | 聚类算法                                          |
|     |      | 监督学习                                                                      | 非监督学习                                         |
|     |      | 喂给它的数据集是带label的数据，已经是完全正确的数据                                              | 喂给它的数据集是无label的数据，是杂乱无章的，经过聚类后才变得有点顺序，先无序，后有序 |
|     |      | 没有明显的前期训练过程，属于memory-based learning(基于记忆学习)                               | 有明显的前期训练过程                                    |
|     | K的含义 | 来了一个样本x，要给它分类，即求出它的y，就从数据集中，在x附近找离它最近的K个数据点，这K个数据点，类别c占的个数最多，就把x的label设为c | K是人工固定好的数字，假设数据集合可以分为K个簇，由于是依靠人工定好，需要一点先验知识   |
| 相似点 |      | 都包含这样的过程：给定一个点，在数据集中找离它最近的点。即二者都用到了NN(Nearest Neighbor)算法，一般用KD树来实现NN。    |                                               |

## 11.2 降维

定义：把一个多因素问题转化成一个较少因素(降低问题的维数)问题，而且较容易进行合理安排，找到最优点或近似最优点，以期达到满意的试验结果的方法。

### 11.2.1 PCA(主成分分析)

PCA 降维的原理：

无监督的降维(无类别信息)-->选择方差大的方向投影,方差越大所含的信息量越大,信息损失越少.可用于特征提取和特征选择。

$$Cov(X, Y) = E[(X - E[X])(Y - E[Y])]$$

PCA 的计算过程： $Cov(X, Y) = E[XY] - E[X]E[Y]$

1. 去平均值，即每一位特征减去各自的平均值
2. 计算协方差矩阵
3. 计算协方差矩阵的特征值与特征向量用(SVD, SVD 比直接特征值分解计算量小)
4. 对特征值从大到小排序
5. 保留最大的个特征向量
6. 将数据转换到个特征向量构建的新空间中

### PCA 推导

中心化后的数据在第一主轴 u1 方向上分布散的最开，也就是说在 u1 方向上的投

影的绝对值之和最大（即**方差最大**），计算投影的方法就是将  $x$  与  $u_1$  做内积，由于只要求  $u_1$  的方向，所以设  $u_1$  是单位向量。

|                                                                                                                                                     |                                                                                                     |                                                                     |                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 最大化：                                                                                                                                                | $\frac{1}{n} \sum_{i=1}^n  \vec{x}_i \cdot \vec{u}_1 $                                              | 也即最大化：                                                              | $\frac{1}{n} \sum_{i=1}^n  \vec{x}_i \cdot \vec{u}_1 ^2 = \frac{1}{n} \sum_{i=1}^n (\vec{x}_i \cdot \vec{u}_1)^2$ |
| 两个向量做内积可以转化成 <b>矩阵乘法</b> ： $\vec{x}_i \cdot \vec{u}_1 = x_i^T u_1$                                                                                  |                                                                                                     |                                                                     |                                                                                                                   |
| 目标函数可以表示为： $\frac{1}{n} \sum_{i=1}^n (x_i^T u_1)^2 = \frac{1}{n} \sum_{i=1}^n u_1^T x_i x_i^T u_1 = \frac{1}{n} u_1^T (\sum_{i=1}^n x_i x_i^T) u_1$ |                                                                                                     |                                                                     |                                                                                                                   |
| 由于：                                                                                                                                                 | $XX^T = \sum_{i=1}^n x_i x_i^T$                                                                     | 目标函数最后化简：                                                           | $\frac{1}{n} u_1^T XX^T u_1$                                                                                      |
| 目标函数和约束条件构成了一个最大化问题：                                                                                                                                |                                                                                                     | $\begin{cases} \max\{u_1^T XX^T u_1\} \\ u_1^T u_1 = 1 \end{cases}$ |                                                                                                                   |
| 构造 <b>拉格朗日函数</b> ：                                                                                                                                  | $f(u_1) = u_1^T XX^T u_1 + \lambda(1 - u_1^T u_1)$                                                  |                                                                     |                                                                                                                   |
| 对 $u_1$ 求导：                                                                                                                                         | $\frac{\partial f}{\partial u_1} = 2XX^T u_1 - 2\lambda u_1 = 0 \rightarrow XX^T u_1 = \lambda u_1$ |                                                                     |                                                                                                                   |
| 将上式代入目标函数表达式即可得到：                                                                                                                                   |                                                                                                     | $u_1^T XX^T u_1 = \lambda u_1^T u_1 = \lambda$                      |                                                                                                                   |
| 取 <b>最大的那个特征值</b> ，那么得到的目标值就最大                                                                                                                      |                                                                                                     |                                                                     |                                                                                                                   |

#### 11.2.1.1 PCA 其优化目标是什么？

最大化投影后方差 + 最小化到超平面距离

#### 11.2.1.2 PCA 白化是什么？

通过 **pca** 投影以后（消除了特征之间的相关性），在各个坐标上除以方差（**方差归一化**）。

### 11.2.2 SVD(奇异值分解)

定义：有一个  $m \times n$  的实数矩阵  $A$ ，我们想要把它分解成如下的形式  $A = U \Sigma V^T$

其中  $U$  和  $V$  均为**单位正交阵**，即有  $UU^T = I$  和  $VV^T = I$ ， $U$  称为左奇异矩

阵， $V$  称为右奇异矩阵， $\Sigma$  仅在**主对角线上有值**，称它为**奇异值**，其它元素均为

0。上面矩阵的维度分别为  $U \in R^{m \times m}$ ， $\Sigma \in R^{m \times n}$ ， $V \in R^{n \times n}$ 。

#### 11.2.2.1 为什么要用 SVD 进行降维？

##### 1.内存少

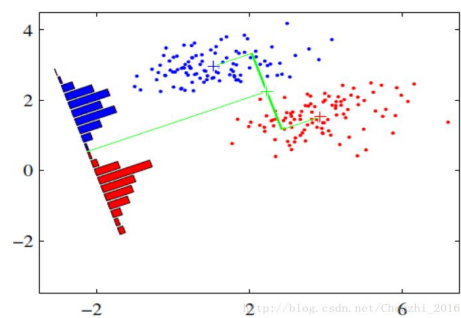
奇异值分解矩阵中奇异值从大到小的顺序减小的特别快，前 10% 甚至 1% 的奇异值的和就占了全部的奇异值之和的 99% 以上。

2.SVD 可以获取另一个方向上的**主成分**，而基于特征值分解的只能获得单个方向上的主成分。

##### 3.数值稳定性

通过 SVD 可以得到 PCA 相同的结果，但是 SVD 通常比直接使用 PCA 更稳定。PCA 需要计算  $XTX$  的值，对于某些矩阵，求协方差时很可能会丢失一些精度。

### 11.2.2 LDA(线性判别式分析)



#### LDA 降维的原理:

LDA 一种有监督的降维算法,它是将高维数据投影到低维上,并且要求投影后的数据具有较好的分类.(也就是说同一类的数据要去尽量投影到同一个簇中去)

投影后的类别内的方差小,类别间的方差较大.

理解: 数据投影在低维度空间后,投影点尽可能的接近,而不同类别的投影点群集的中心点彼此之间的离得尽可能大。

### 11.2.3 PCA vs SVD

|           | PCA                                                      | SVD            |
|-----------|----------------------------------------------------------|----------------|
| 都是矩阵分解的技术 | 对协方差矩阵操作后分解PCA                                           | 直接分解SVD        |
|           | 奇异值和特征向量存在关系, 即有: $\lambda_i = s_i^2 / (n - 1)$          |                |
|           | 只能获得单个方向上的主成分, 与SVD的右奇异向量的压缩效果相同                         | 可以获取另一个方向上的主成分 |
|           | 通过SVD可以得到PCA相同的结果, 但是SVD通常比PCA更稳定; 因为在PCA求协方差时很可能会丢失一些精度 |                |

### 11.2.4 LDA vs PCA

|    | LDA                                       | PCA                       |
|----|-------------------------------------------|---------------------------|
| 区别 | 有监督                                       | 无监督                       |
|    | 降维最多降到类别数 $K-1$ 的维数                       | 无这个限制                     |
|    | LDA更 <b>依赖均值</b> ，如果样本信息更依赖方差的话，效果将没有PCA好 |                           |
|    | 选择分类性能最好的投影方向                             | 选择样本点投影具有 <b>最大方差</b> 的方向 |
|    | LDA可能会过拟合数据                               |                           |
|    | 还可以用于 <b>分类</b>                           |                           |
| 联系 | 降维时均使用了 <b>矩阵特征分解</b> 的思想                 |                           |
|    | 都假设符合 <b>高斯分布</b>                         |                           |

### 11.2.5 降维的作用是什么？

- ①降维可以缓解维度灾难问题
- ②降维可以在压缩数据的同时让信息损失最小化
- ③理解几百个维度的数据结构很困难，两三个维度的数据通过可视化更容易理解

### 11.2.6 矩阵的特征值和特征向量的物理意义是什么？

对于一个非方阵的矩阵  $A$ ，它代表一个多维空间里的多个数据。求这个矩阵  $A$  的协方差阵  $Cov(A, A')$ ，得到一个方阵  $B$ ，求的特征值，就是求  $B$  的特征值，它就是代表矩阵  $A$  的那些数据，在那个多维空间中，各个方向上分散的一个度量（即理解为它们在各个方向上的特征是否明显，特征值越大，则越分散，也就是特征越明显），而对应的特征向量：是它们对应的各个方向。

（协方差：用于衡量两个变量的总体误差。而方差是协方差的一种特殊情况，即当两个变量是相同的情况。）

### 维度灾难是什么？为什么要关心它？

当特征向量数很少时，增加特征，可以提高算法的精度，但当特征向量的维数增加到一定数量之后，再增加特征，算法的精度反而会下降。

## 12. 概率模型

### 12.1 朴素贝叶斯

是一个生成模型，其次它通过学习已知样本，计算出联合概率，再求条件概率。

**原理：**基于贝叶斯定理与特征条件独立假设的分类方法。对于给定的待分类项  $x$ ，通过学习到的模型计算后验概率分布，即：在此项出现的条件下各个目标类别出现的概率，将后验概率最大的类作为  $x$  所属的类别。

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

贝叶斯定理的公式表达式：

#### 12.1.2 为什么朴素贝叶斯如此“朴素”？

在计算条件概率分布  $P(X=x | Y=C_k)$  时，NB 引入了一个很强的条件独立假设，即，当  $Y$  确定时， $X$  的各个特征分量取值之间**相互独立**。

### 12.1.3 朴素贝叶斯的优缺点？

|    |                                          |
|----|------------------------------------------|
| 优点 | 1. 有稳定的分类效率                              |
|    | 2. 对于小规模的数据表现很好，能处理多分类问题，适合 <b>增量式训练</b> |
|    | 3. 对缺失数据不太敏感，算法比较简单，常用于文本分类。             |
| 缺点 | 1. 对输入数据的表达形式很敏感（离散、连续，极大值极小值）           |
|    | 2. <b>特征条件独立性假设</b> 在实际应用中往往是不成立的：       |
|    | 在属性个数比较多或者属性之间相关性较大时，分类效果不好；             |
|    | 在属性 <b>相关性较小时</b> ，朴素贝叶斯性能最为良好。          |

### 12.1.4 为什么引入条件独立性假设？

为了避免贝叶斯定理求解时面临的组合爆炸、样本稀疏问题。

### 12.1.5 在估计条件概率 $P(X|Y)$ 时出现概率为 0 的情况怎么办？

采用**贝叶斯估计**。即引入  $\lambda$ ，当  $\lambda=1$  时，就是普通的极大似然估计； $\lambda=1$  时称为拉普拉斯平滑。

### 12.1.6 为什么属性独立性假设在实际情况中很难成立，但 NB 仍能取得较好效果？

使用分类器之前，首先做的第一步往往是**特征选择**，目的就是为了排除特征之间的共线性、选择相对较为独立的特征；对于与分类任务来说，只要各类别的条件概率排序正确，无需精准概率值就可以导致正确分类；

如果属性间依赖对所有类别影响相同，或依赖关系的影响能相互抵消，则属性条件独立性假设在降低计算复杂度的同时不会对性能产生负面影响。

如何对贝叶斯网络进行采样？

没有观测变量：

采用祖先采样，核心思想是根据有向图的顺序，先对祖先节点进行采样，只有当某个节点的父节点都已经完成采样，才对该节点进行采样。

只对非观测变量采样，但是最终得到的样本需要赋一个重要性权值，这种采样方法称作似然加权采样；

还可以用 MCMC 采样法来进行采样

## 12.2 朴素贝叶斯 vs LR

|    | 朴素贝叶斯                                      | LR                           |
|----|--------------------------------------------|------------------------------|
| 区别 | 生成模型                                       | 判别模型                         |
|    | 根据已有样本进行贝叶斯估计学习出先验概率 $P(Y)$ 和条件概率 $P(X Y)$ | 根据极大化对数似然函数直接求出条件概率 $P(Y X)$ |
|    | 基于很强的条件独立假设（在已知分类 $Y$ 的条件下，各个特征变量取值是相互独立的） | 没有要求                         |
|    | 适用于数据集少                                    | 适用于大规模数据集                    |

## 二、深度学习

什么是深度学习？深度学习的训练过程是什么？

无监督预训练+有监督微调（fine-tune）

过程：（1）自下而上非监督学习特征 （2）自顶向下有监督微调

深度学习与机器学习有什么区别？

机器学习在训练模型之前，需要手动设置特征，即需要做特征工程；

深度学习可自动提取特征；所以深度学习自动提取的特征比机器学习手动设置的特征鲁棒性更好；

### 13.机器学习

#### 13.1 你是怎么理解偏差和方差的平衡的？

偏差是真实值和预测值之间的偏离程度；方差是预测值得分散程度，即越分散，方差越大；

**13.2** 给你一个有 1000 列和 1 百万行的训练数据集，这个数据集是基于分类问题的。经理要求你来降低该数据集的维度以减少模型计算时间，但你的机器内存有限。你会怎么做？

处理方法：

1.由于我们的 RAM 很小，首先要关闭机器上正在运行的其他程序，包括网页浏览器等，以确保大部分内存可以使用。

2.随机采样数据集：可以创建一个较小的数据集，比如有 1000 个变量和 30 万行，然后做计算。

3.为了降低维度，可以把数值变量和分类变量分开，同时删掉相关联的变量。对于数值变量，将使用相关性分析；对于分类变量，可以用卡方检验(统计学)。

4.另外，还可以使用 PCA，并挑选可以解释在数据集中有最大偏差的成分。

5.利用在线学习算法，如 VowpalWabbit。

6.利用 SGD 建立线性模型也很有帮助。

**13.3** 给你一个数据集，这个数据集有缺失值，且这些缺失值分布在离中值有 1 个标准偏差的范围内。百分之多少的数据不会受到影响？为什么？

约有 32%的数据将不受缺失值的影响。由于数据分布在中位数附近，先假设这是一个正态分布。在一个正态分布中，约有 68%的数据位于跟平均数（或众数、

中位数) 1 个标准差范围内, 那么剩下的约 32% 的数据是不受影响的。

### 13.4 模型受到低偏差和高方差问题时, 应该使用哪种算法来解决问题呢?

可以使用 bagging 算法(随机森林)。低偏差意味着模型的预测值接近实际值, 即该模型有足够的灵活性, 以模仿训练数据的分布。

bagging 算法把数据集分成重复随机取样形成的子集。然后这些样本利用单个学习算法生成一组模型。接着, 利用投票(分类)或平均(回归)把模型预测结合在一起。

另外, 为了应对大方差, 我们可以:

1. 使用正则化技术, 惩罚更高的模型系数, 从而降低了模型的复杂性。
2. 使用可变重要性图表中的前 n 个特征。可以用于当一个算法在数据集中的所有变量里很难寻找到有意义信号的时候。

### 13.5 怎么理解偏差方差的平衡的?

偏差误差在量化平均水平之上, 预测值跟实际值相差多远时有用。

**高偏差**误差意味着我们的模型表现不太好, 因为没有抓到重要的趋势。

而另一方面, 方差量化了在同一个观察上进行的预测是如何彼此不同的。

**高方差**模型会过度拟合你的训练集, 而在训练集以外的数据上表现很差。

### 13.6 协方差和相关性有什么区别?

相关性是协方差的标准化的格式。协方差本身很难做比较。如: 计算工资(\$ ) 和年龄(岁) 的协方差, 因为这两个变量有不同的度量, 所以会得到不能做比较的不同的协方差。

$$\Sigma_{ij} = \text{cov}(X_i, X_j) = E[(X_i - \mu_i)(X_j - \mu_j)] = E[X_i X_j] - \mu_i \mu_j$$

为了解决这个问题, 通过计算相关性来得到一个介于-1 和 1 之间的值, 就可以忽略它们各自不同的度量。

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y}$$

### 13.7 把分类变量当成连续型变量会更得到一个更好的预测模型吗?

只有在分类变量在本质上是有序的情况下才可以被当做连续型变量来处理。

“买了这个的客户, 也买了.....” 亚马逊的建议是哪种算法的结果?

### 13.8 机器学习中分类器指的是什么?

指输入离散或连续特征值的向量, 并输出单个离散值或者类型的系统。

对统计这一块了解吗? p 值是什么?

当原假设为真时所得到的样本观察结果或更极端结果出现的概率;

如果 P 值很小, 说明 **bai** 原假设情况的发生的概率很小;

如果出现了, 根据小概率原理, 我们就有理由拒绝原假设, P 值越小, 我们拒绝原假设的理由越充分。

### 13.10 请简要说说一个完整机器学习项目的流程?

|                |                            |
|----------------|----------------------------|
| <b>1. 定义问题</b> | 导入类库和数据集                   |
| <b>2. 理解数据</b> | 2.1 通过描述性统计来分析数据           |
|                | 2.2 通过可视化来观察数据             |
| <b>3. 数据准备</b> | 3.1 数据清洗 3.2 特征选择 3.3 数据转换 |
| <b>4. 评估算法</b> | 4.1 分离数据集：以便于验证模型          |
|                | 4.2 定义模型评估标准：用来评估算法模型      |
|                | 4.3 算法审查：抽样审查线性算法和非线性算法    |
| <b>5. 优化模型</b> | 5.1 算法调参： 得到最佳结果           |
|                | 5.2 集成算法： 提高算法模型的准确度       |
| <b>6. 结果部署</b> | 6.1 预测评估数据集                |
|                | 6.2 利用整个数据集生成模型            |
|                | 6.3 序列化模型                  |